



中国科学技术大学
University of Science and Technology of China

计算系统概论A

Introduction to Computing Systems
(CS1002A.03)

Chapter 8 Data Structures

陈俊仕

cjuns@ustc.edu.cn

2025 Fall

计算机科学与技术学院

School of Computer Science and Technology

1 Review

2 Subroutines

3 Control Instructions for Subroutines

4 Memory Model for Program Execution

5 The Stack

6 Implementing Functions in C

1

Review

2

Subroutines

3

Control Instructions for Subroutines

4

Memory Model for Program Execution

5

The Stack

6

Implementing Functions in C

Control Instructions for Subroutines



	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BR	0	0	0	0	n	z	p	PCoffset9								
JSR	0	1	0	0	1	PCoffset11										
JSRR	0	1	0	0	0	0	0	BaseR		0	0	0	0	0	0	0
RTI	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
JMP	1	1	0	0	0	0	0	BaseR		0	0	0	0	0	0	0
RET	1	1	0	0	0	0	0	1	1	1	0	0	0	0	0	0
TRAP	1	1	1	1	0	0	0	0	TrapVector8							

★8.15 A program encounters a breakpoint and halts. The computer operator does not change the state of the computer in any way but immediately presses the **run** button to resume execution. The following table shows the contents of MAR and MDR for the first nine memory accesses that the LC-3 performs after resuming execution. Your job: Fill in the missing entries.

	MAR	MDR
1st:		x5020
2nd:		xF0F0
3rd:		
4th:	x2000	x020A
5th:		x040A
6th:		x61FE
7th:		
8th:		xC1C0
9th:	x4002	xF025

7.34 It is often useful to find the midpoint between two values. For this problem, assume A and B are both even numbers, and A is less than B. For example, if $A = 2$ and $B = 8$, the midpoint is 5. The following program finds the midpoint of two even numbers A and B by continually incrementing the smaller number and decrementing the larger number. You can assume that A and B have been loaded with values before this program starts execution.
Your job: Insert the missing instructions.

```

                .ORIG  x3000
LD              R0,A
LD              R1,B
X              ----- (a)
                ----- (b)
ADD            R2,R2,R1
                ----- (c)
ADD            R1,R1,#-1
                ----- (d)
BRnzp         X
DONE          ST      R1,C
TRAP          x25
A             .BLKW   1
B             .BLKW   1
C             .BLKW   1
                .END

```

★7.35 The program stored in memory locations x3000 to x3007 loads a value from memory location x3100, then does some processing, and then stores a result in memory location x3101. Following is an incomplete specification of the program. Your job: Complete the specification of the program.

Address	Contents	Assembly code
x3000	0101 001 001 1 00000	AND R1, R1, #0
x3001	0010 000	LD R0, x3100
x3002	0000 110 000000011	BRnz x3006
x3003	0001	ADD
x3004		
x3005	0000 111	
x3006	0011 001	ST R1, x3101
x3007	1111 0000 0010 0101	HALT

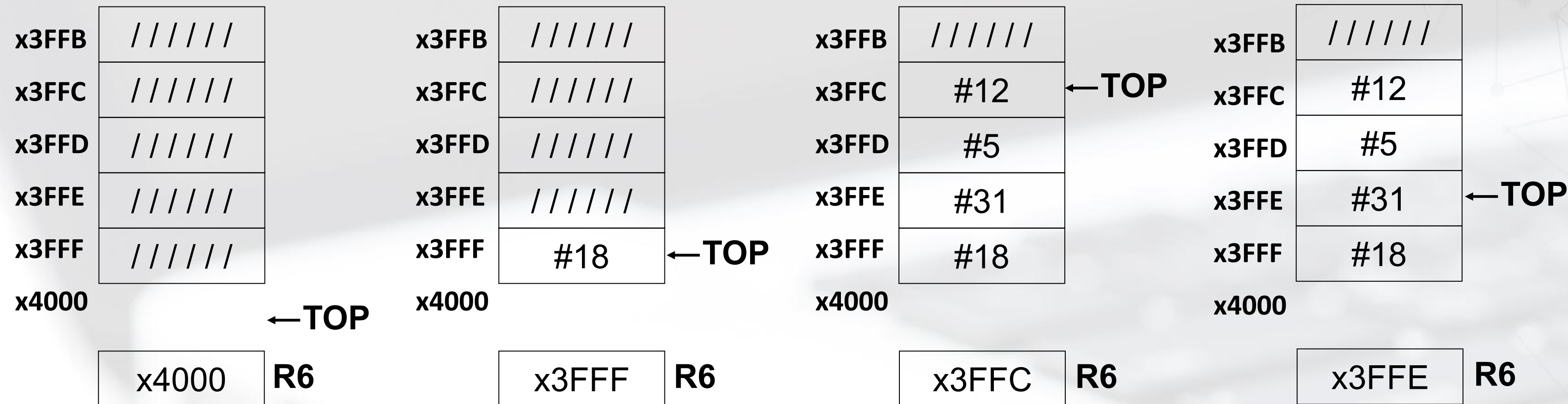
instruction boundaries thereafter, that is, after a number of instructions executed. Part of the snapshot is shown below. Your job is to complete the snapshot. Note that the program enters the TRAP x25 service routine after executing 17 instructions. Therefore, some instructions must execute more than once.

Note that in the following table, some entries are designated xxxx. You do not have to fill in those entries. Also, you can ignore snapshots for any instructions that are not listed in the table.

To help you in this process, we have taken a snapshot of part of the state of the machine before the first instruction executes and at several

Instruction #	PC	MAR	MDR	R0	R1
Initial	x3000	xxxx	xxxx	xxxx	xxxx
1	x3001	xxxx	xxxx	xxxx	
2	x3002				
3	x3003	xxxx	xxxx		
4	x3004		x1240		
5	x3005	xxxx	xxxx	x0002	
9	x3005	xxxx	xxxx	x0001	
13	x3005	xxxx	xxxx	x0000	
14	x3002	xxxx	xxxx		
15	x3006	xxxx	xxxx		
16	x3007				
17	xxxx	xxxx	xxxx		

Basic Push and Pop Code



PUSH

```
ADD R6, R6, #-1           ; increment stack ptr
STR R0, R6, #0            ; store data(R0) to TOS
```

POP

```
LDR R0, R6, #0            ; load data(R0) from TOS
ADD R6, R6, #1            ; decrement stack ptr
```

■ **Note:** Stacks can grow in either direction (toward higher address or toward lower addresses)

Arithmetic Using a Stack

■ Instead of registers, some ISA's use a stack for source and destination operations: a **zero-address** machine.

● Example: ADD instruction pops two numbers from the stack, adds them, and pushes the result to the stack.

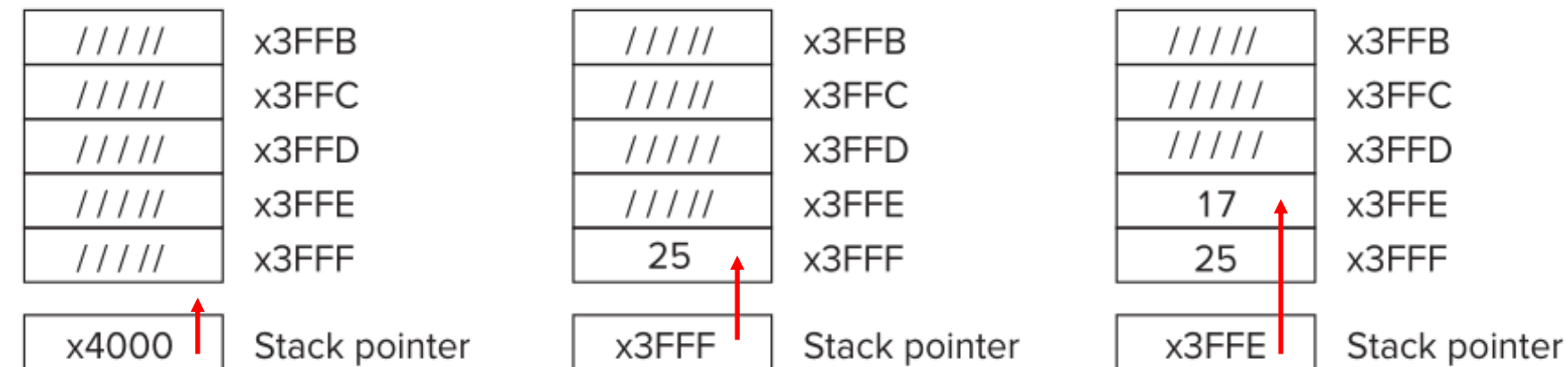
ADD vs. **ADD R0,R1,R2**

■ Evaluating $(A+B) \cdot (C+D)$ using a stack:

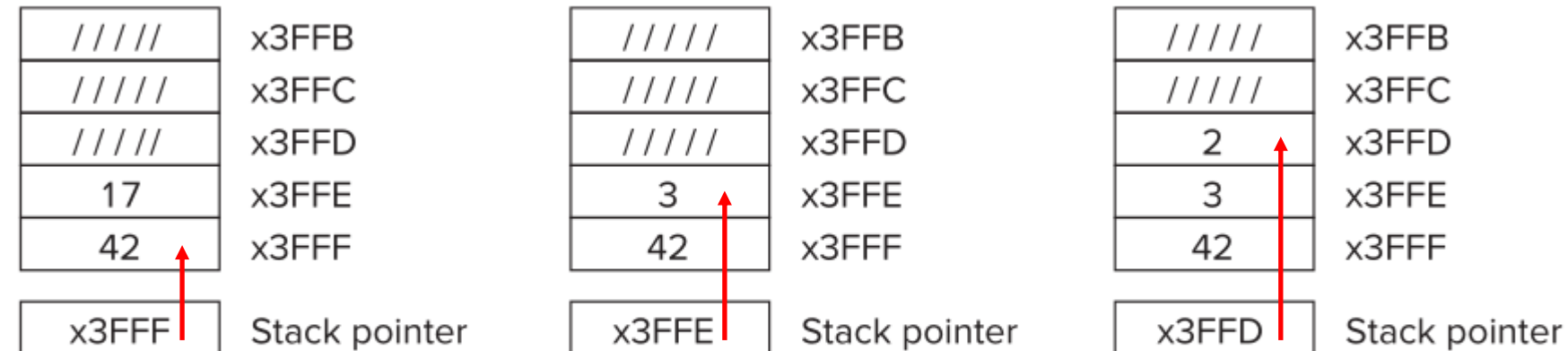
```
push A
push B
ADD
push C
push D
ADD
MULTIPLY
pop result
```



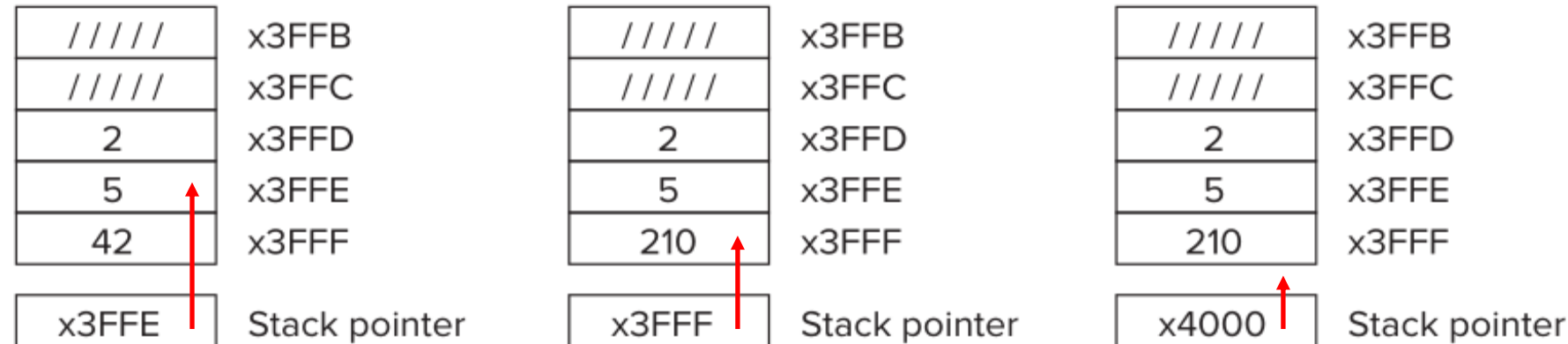
$(25+17) \times (3+2)$



(a) Before (b) After first push (c) After second push



(d) After first add (e) After third push (f) After fourth push



(g) After second add (h) After multiply (i) After pop

8.3 The LC-3 ISA has been augmented with the following push and pop instructions. Push Rn pushes the value in Register n onto the stack. Pop Rn removes a value from the stack and loads it into Rn. The following figure shows a snapshot of the eight registers of the LC-3 BEFORE and AFTER the following six stack operations are performed. Identify (a)–(d).

BEFORE				AFTER			
R0	x0000	PUSH R4	R0	x1111			
R1	x1111	PUSH (a)	R1	x1111			
R2	x2222	POP (b)	R2	x3333			
R3	x3333	PUSH (c)	R3	x3333			
R4	x4444	POP R2	R4	x4444			
R5	x5555	POP (d)	R5	x5555			
R6	x6666		R6	x6666			
R7	x7777		R7	x4444			

6.15 Shown below are the contents of memory and registers **before** and **after** the LC-3 instruction at location x3010 is executed. Your job: Identify the instruction stored in x3010. *Note:* There is enough information below to uniquely specify the instruction at x3010.

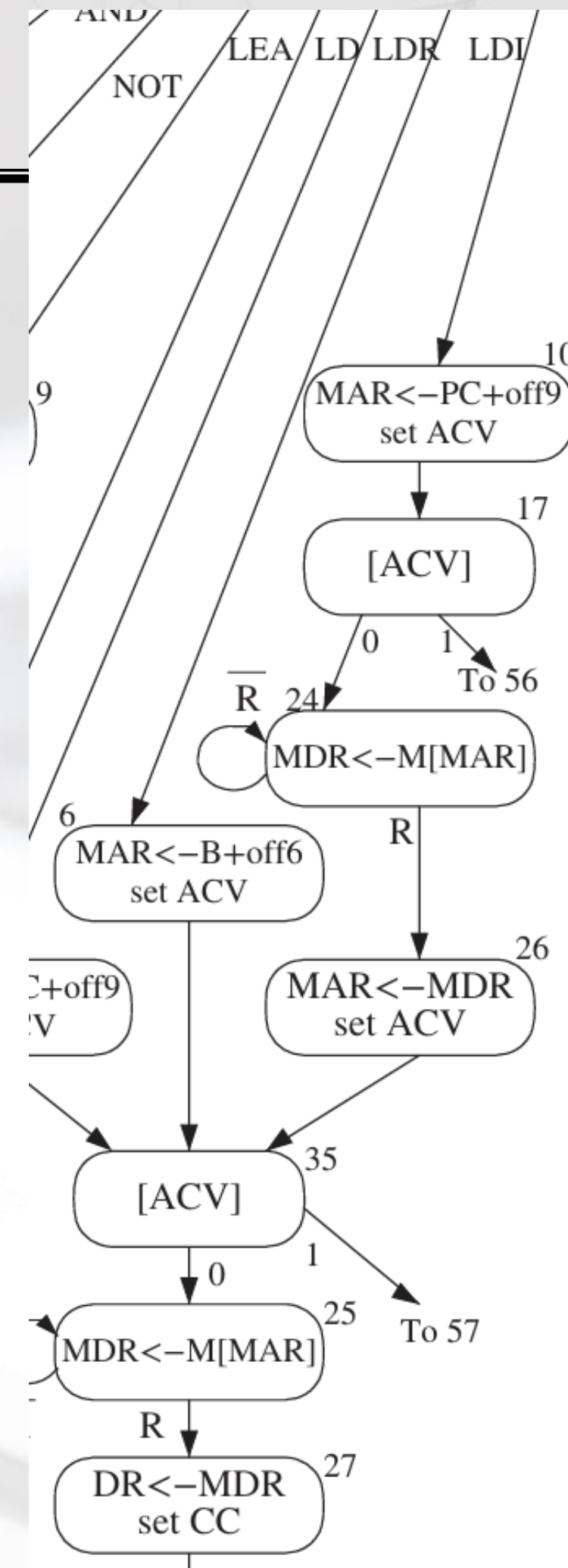
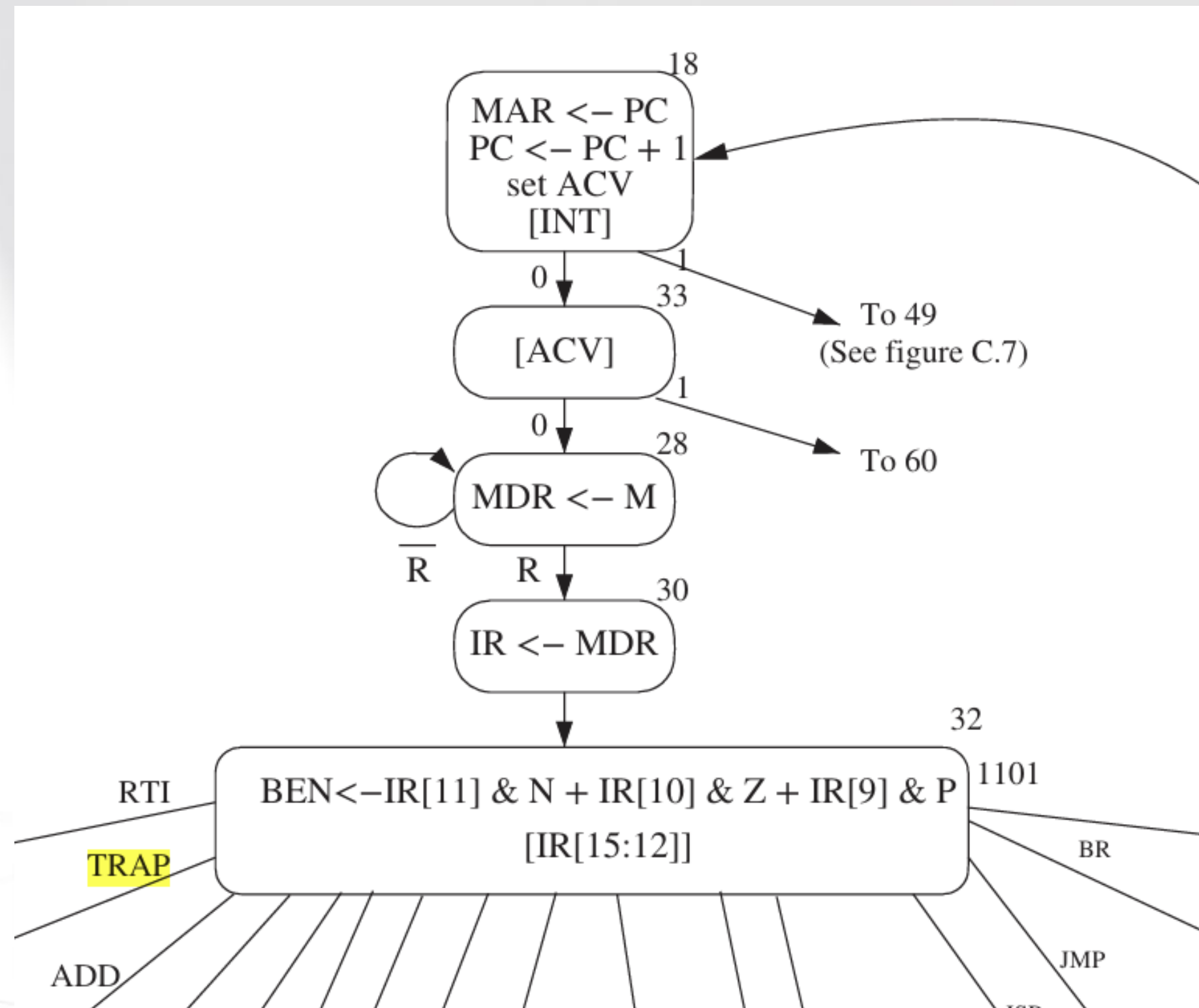
	Before	After
R0:	x3208	x3208
R1:	x2d7c	x2d7c
R2:	xe373	xe373
R3:	x2053	x2053
R4:	x33ff	x33ff
R5:	x3f1f	x3f1f
R6:	xf4a2	xf4a2
R7:	x5220	x5220

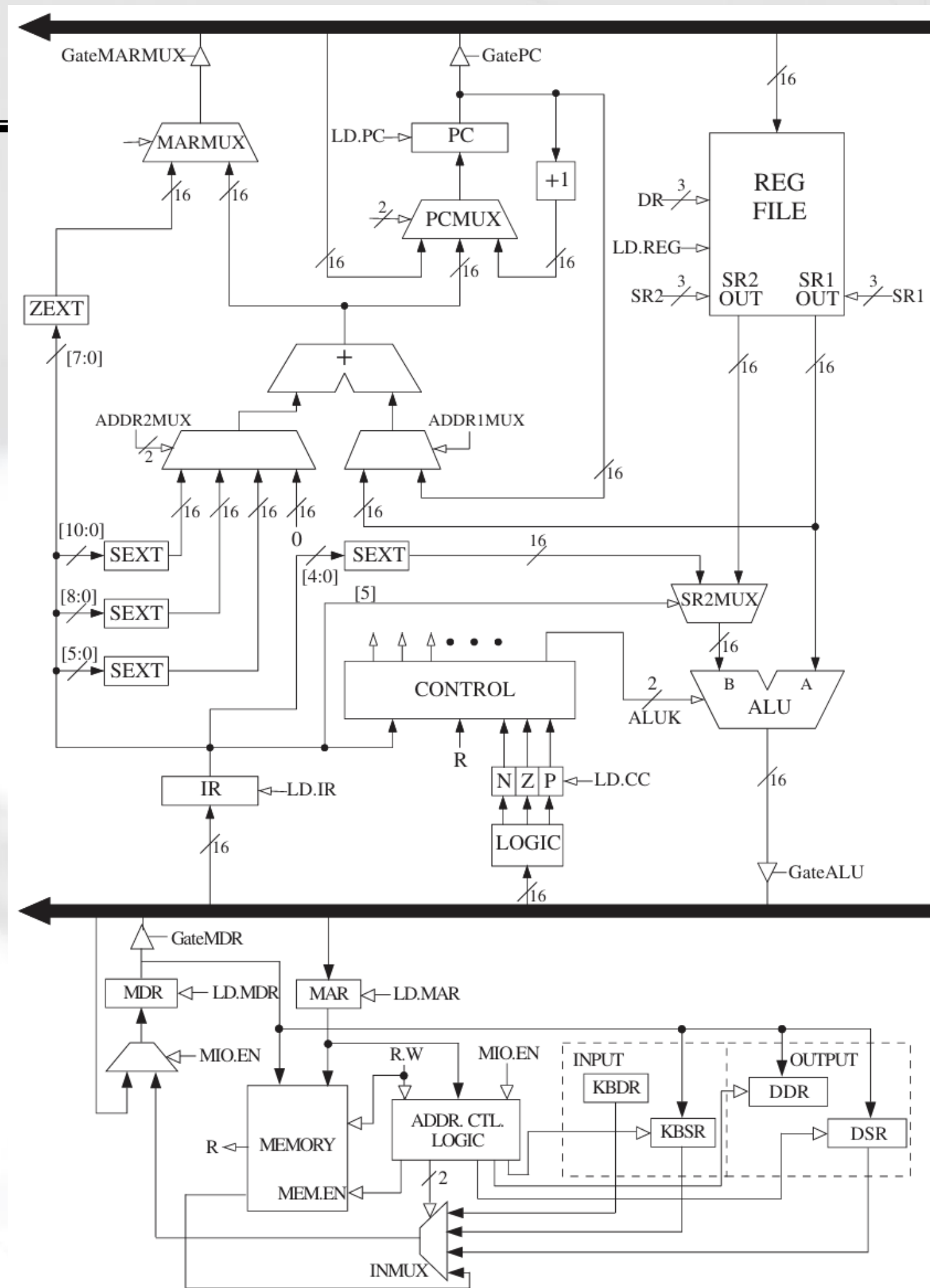
...		
x3400:	x3001	x3001
x3401:	x7a00	x7a00
x3402:	x7a2b	x7a2b
x3403:	xa700	xa700
x3404:	xf011	xf011
x3405:	x2003	x2003
x3406:	x31ba	xe373
x3407:	xc100	xc100
x3408:	xefef	xefef
...		

★6.26 During the execution of an LC-3 program, an instruction in the program starts executing at clock cycle T and requires 15 cycles to complete. The table lists **ALL** five clock cycles during the processing of this instruction, which require use of the bus. The table shows for each of those clock cycles: which clock cycle, the state of the state machine, the value on the bus, and the important control signals that are active during that clock cycle.

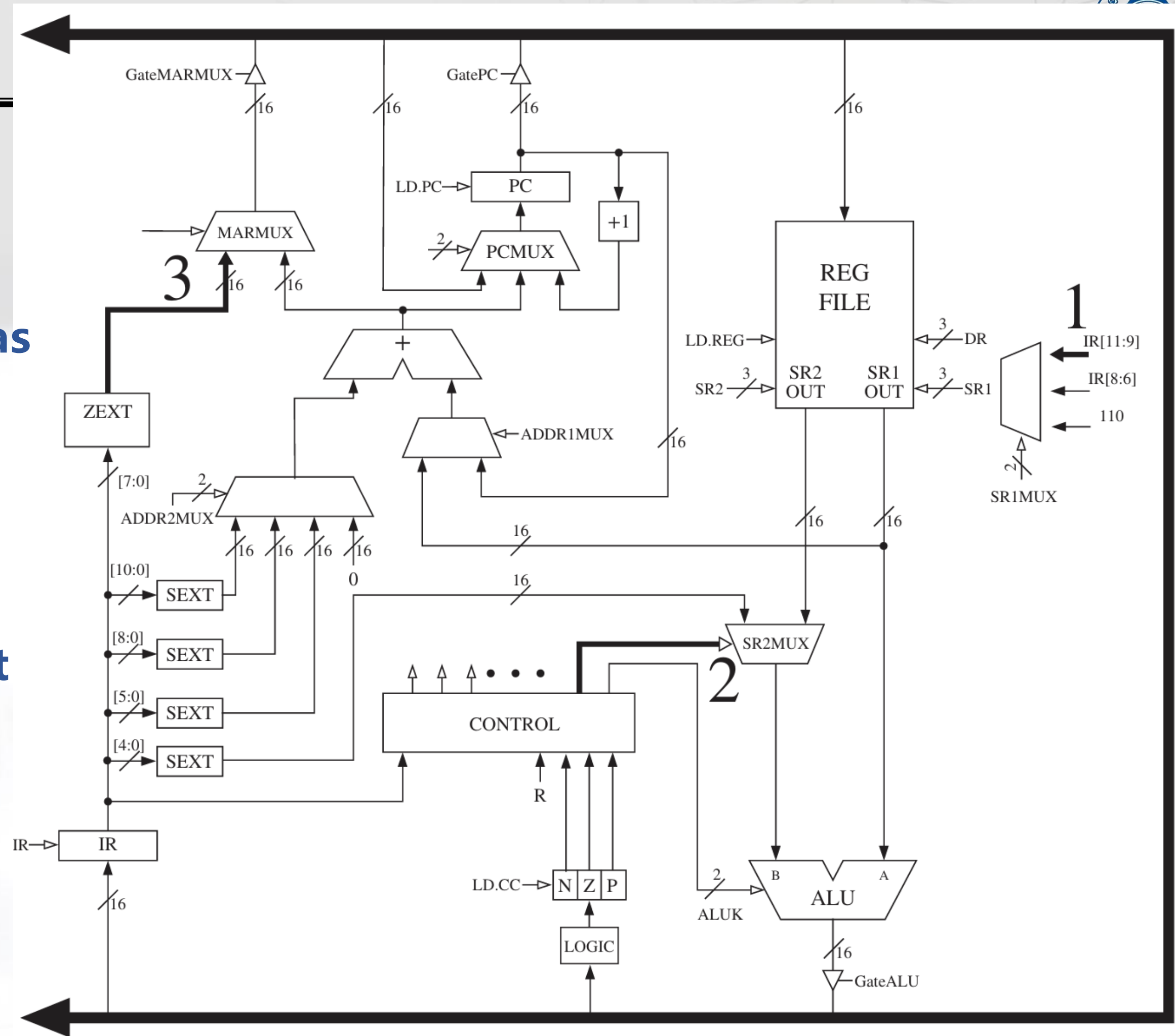
Cycle	State	Bus	Important Control Signals for This Cycle
T	18	x3010	LD.MAR = 1, LD.PC = 1, PCMux = PC + 1, GatePC = 1
T + 4			
T + 6		x3013	
T + 10		x4567	
T + 14		x0000	LD.REG = 1, LD.CC = 1, GateMDR = 1, DR = 001

- Fill in the missing entries in the table.
- What is the instruction being processed?
- Where in memory is that instruction?
- How many clock cycles does it take memory to read or write?
- There is enough information above for you to know the contents of three memory locations. What are they, and what are their contents?





- 1. What opcodes use IR[11:9] as inputs to SR1?
- 2. Where does the control signal of this mux come from?
- 3. What opcodes use this input to the MARMUX?



5.60

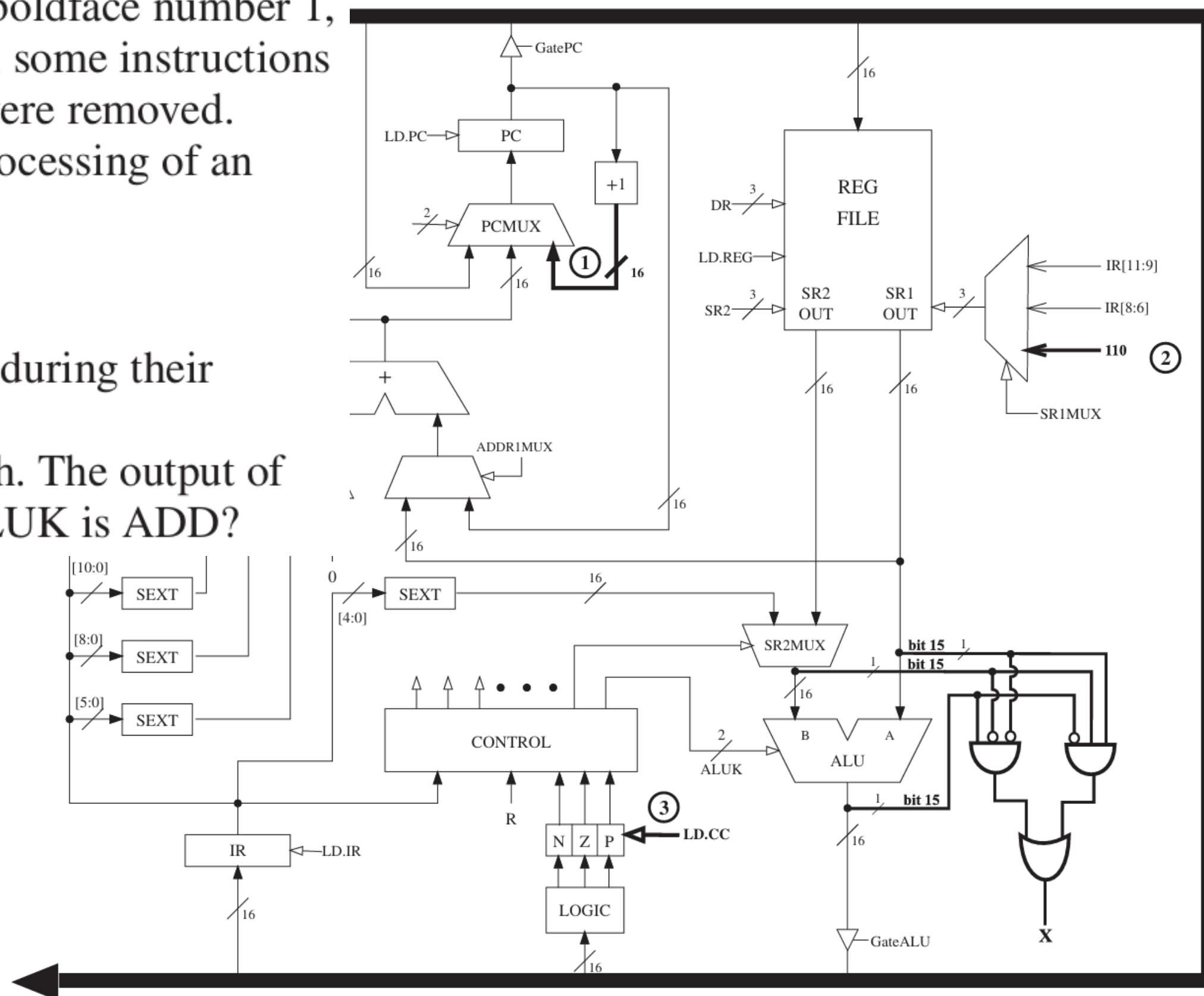


Note that several of the lines (i.e., signals) in the LC-3 data path have been drawn in boldface. Each line is designated by a boldface number 1, 2, or 3. Not all instructions use all three lines. That is, some instructions would not function correctly if the line (i.e., signal) were removed.

List the opcodes that utilize line 1 during their processing of an instruction.

List the opcodes that require LD.CC=1 on line 3 during their processing of an instruction.

Note the logic (in boldface) added to the data path. The output of that logic is labeled X. What does X=1 indicate if ALUK is ADD?



5.59 Every LC-3 instruction takes eight cycles to be fetched and decoded, if we assume every memory access takes five cycles. The total number of cycles an LC-3 instruction takes to be completely processed, however, depends on what has to be done for that instruction.

Assuming every memory access takes five cycles, and assuming the LC-3 processes one instruction at a time, from beginning to end, how many clock cycles does each instruction take? For each instruction, how many cycles are required to process it?

Instruction	Number of cycles
ADD	
AND	
LD	
LEA	
LDI	
NOT	
BRnzp	
TRAP	

5.56 This problem tests your knowledge of the instruction cycle for processing the NOT instruction. You are asked to show the values of several control signals in every clock cycle of the sequence that is used to process the NOT instruction.

The instruction cycle starts with state 18 as shown in the following table. Your job: Identify each state in the sequence, and show the values of the control signals listed during each state in the sequence. Use the convention specified below. For a particular state, if the value of a control signal does not matter, fill it with an X. You may not have to use all the rows.

Note: Assume a memory access takes one clock cycle.

Cycle	State	LD.PC	LD.MAR	LD.MDR	LD.REG	LD.CC	GateALU	GatePC	ALUK	PCMUX
1	18									
2										
		LD.PC	0: load not enabled 1: load enabled				GateALU	0: do not pass signal 1: pass signal		
		LD.MAR	0: load not enabled 1: load enabled				GatePC	0: do not pass signal 1: pass signal		
		LD.MDR	0: load not enabled 1: load enabled				ALUK	00: ADD 01: AND 10: NOT 11: Pass input A		
		LD.REG	0: load not enabled 1: load enabled				PCMUX	00: PC+1 01: BUS 10: from adder		
		LD.CC	0: load not enabled 1: load enabled							

★7.33 We have a program with some missing instructions, and we have a table consisting of some information and some missing information associated with five specific clock cycles of the program's execution. Your job is to complete both!

Insert the missing instructions in the program and the missing information in the table. Cycle numbering starts at 1. That is, cycle 1 is the first clock cycle of the processing of LD R0,A. Note that we have not said anything about the number of clock cycles a memory access takes. You do have enough information to figure that out for yourself. Note that we are asking for the value of the registers DURING each clock cycle.

```

.ORIG x3000
LD  R0, A
LD  R1, B
NOT R1, R1
ADD R1, R1, #1
AND R2, R2, #0
AGAIN ----- (a)
----- (b)
BRnzp AGAIN
DONE ST  R2, C
      HALT
A    .FILL #5
B    .FILL ----- (c)
C    .BLKW #1
      .END

```

Cycle Number	State Number	Information			
		LD.REG: 1	DRMUX:	GateMDR:	
		LD.CC:	GateALU:	GatePC:	
16	35	LD.MDR:	MDR:	IR:	
		LD.IR:			
50		LD.REG: 1	MDR: x_4A_	DRMUX:	
		BUS: x0001		GateMDR:	
57	1	PC:	IR: x_040	GateALU:	
		BUS: x0003		GatePC:	
	23	ADDR1MUX:	ADDR2MUX:		
		LD.PC: 1	PC: x3008	PCMUX:	ADDER

What is stored in C at the end of execution for the specific operands given in memory locations A and B?

1

Review

2

Subroutines

3

Control Instructions for Subroutines

4

Memory Model for Program Execution

5

The Stack

6

Implementing Functions in C



A C program that uses a function to print a banner message

```
1    #include <stdio.h>
2
3    void PrintBanner();    // Function declaration
4
5    int main(void)
6    {
7        PrintBanner();    // Function call
8        printf("A simple C program.\n");
9        PrintBanner();
10   }
11
12   void PrintBanner()    // Function definition
13   {
14       printf("=====\n");
15   }
```

Function in C

- Smaller, simpler, subcomponent of program

- Provides **abstraction**

- hide low-level details
- give high-level structure to programmer, easier to understand overall program flow
- enables separable, independent development

- **C functions**

- zero or multiple arguments passed in
- single result returned (optional)
- return value is always a particular type

- In other languages, called **procedures, subroutines, methods ...**

Example of High-Level Structure

```
main()  
{  
    SetupBoard();      /* place pieces on board */  
    DetermineSides();  /* choose black/white */  
  
    /* Play game */  
    do {  
        WhitesTurn();  
        BlacksTurn();  
    } while (NoOutcomeYet());  
}
```

Structure of program is evident, even without knowing implementation.

Functions in C

```
double ValueInDollars(double amount, double rate);
```

declaration

```
main()
```

```
{
```

```
...
```

```
dollars = ValueInDollars(francs, DOLLARS_PER_FRANC);
```

```
printf("%f francs equals %f dollars.\n", francs, dollars);
```

```
...
```

```
}
```

definition

```
double ValueInDollars(double amount, double rate)
```

```
{
```

```
    return amount * rate;
```

```
}
```

Functions in C

■ Function Declaration (also called prototype)

```
int Factorial(int n);
```

type of
return value

name of
function

types of all
arguments

■ Function Call -- used in expression

```
a = x + Factorial(f + g);
```

1. evaluate arguments

2. execute function

3. use return value in expression

Function Definition

■ Return type, function name, types of arguments

- must match function declaration
- give name to each argument (doesn't have to match declaration)

```
int Factorial(int n)
{
    int i;
    int result = 1;
    for (i = 1; i <= n; i++)
        result *= i;
    return result;
}
```

← gives control back to calling function and returns value

Implementing Functions in C

- **Functions** in C are the high-level equivalent of **subroutines** at the LC-3 machine level.
- Functions in C are implemented using a similar set of mechanisms as assembly level subroutines.
- There are four basic phases in the execution of a function call:
 - (1) argument values from **the caller** are passed to **the callee**
 - (2) control is transferred to the callee
 - (3) the callee executes its task
 - (4) control is passed back to the caller, along with a return value.
- We will examine how all of this is accomplished on the LC-3.

Calling Function

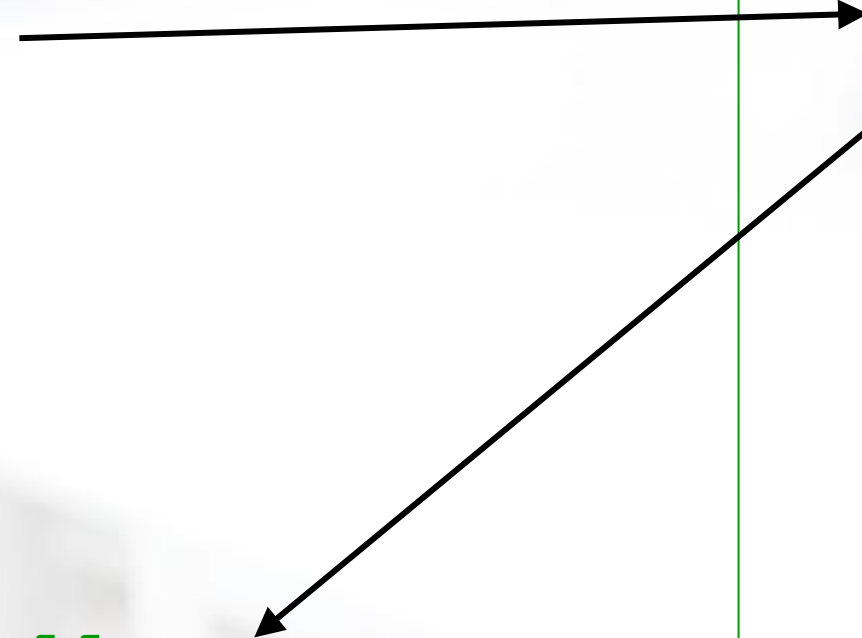
Caller

1. pass arguments
2. call function

3. get result from callee

Callee

1. execute code
2. return



Storage Requirements

- **Code** must be stored in memory so that we can execute the function.
- **Parameters** must be sent from the **caller** to the **callee** so that the function receives them.
- **Local/global variables** for the function must be stored somewhere, is one copy enough?
- **Return address** must be stored so that control can be returned to the **caller**.
- **Return values** must be sent from the **callee** to the **caller**, that's how results are returned.

```
// main program
Int a = 10;
Int b = 20;
Int c = foo(a, b);

Int foo(int x, int y)
{
    Int z;
    z = x + y;
    return z;
}
```

- What needs to be stored?
 - Code, parameters, local/global variables, return address/values

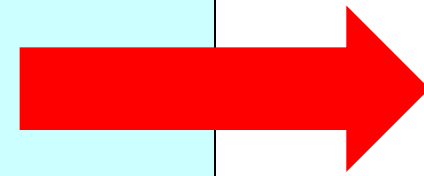
Possible Solution: Mixed Code and Data

Possible Solution: Mixed Code and Data

■ Function implementation:

```
// main program
Int a = 10;
Int b = 20;
Int c = foo(a, b);

Int foo(int x, int y)
{
    Int z;
    z = x + y;
    return z;
}
```



```
foo          BR   foo_begin    ;skip over data
; Memory allocation
foo_rv       .BLKW 1           ;return value
foo_ra       .BLKW 1           ;return address
foo_paramx   .BLKW 1           ;'x' parameter
foo_paramy   .BLKW 1           ;'y' parameter
foo_localz   .BLKW 1           ;'z' local variable

foo_begin    ST   R7, foo_ra    ;save return
...
LD   R7, foo_ra    ;restore return
RET
```

■ Calling sequence

```
ST R1, foo_paramx    ; R1 has 'x'
ST R2, foo_paramy    ; R2 has 'y'
JSR foo              ; Function call
LD R3, foo_rv         ; R3 = return value
```

Possible Solution: Mixed Code and Data

■ Advantages:

- Code and data are close together
- Conceptually easy to understand, code generation is relatively simple
- Minimizes register usage for variables
- Data persists through life of program
- Few instructions are spent moving data

■ Disadvantages:

- Code is vulnerable to self-modification
- Cannot handle recursion or parallel execution
- Consumes resource for inactive functions

Possible Solution: Separate Code and Data

Possible Solution: Separate Code and Data

■ Memory allocation

```
// main program
Int a = 10;
Int b = 20;
Int c = foo(a,b) ;

Int foo(int x,int y)
{
    Int z;
    z= x+y;
    return z;
}
```

```
; Code for foo() and bar() are somewhere else

foo_rv      .BLKW 1      ; foo return value
foo_ra      .BLKW 1      ; foo return address
foo_paramx  .BLKW 1      ; foo 'x' parameter
foo_paramy  .BLKW 1      ; foo 'y' parameter
foo_localz  .BLKW 1      ; foo 'z' local

bar_rv      .BLKW 1      ; bar return value
bar_ra      .BLKW 1      ; bar return address
bar_paramw  .BLKW 1      ; bar 'w' parameter
```

■ Function code call is similar to mixed solution

Possible Solution: Separate Code and Data

■ Advantages:

- Code can be marked 'read only'
- Conceptually easy to understand
- Early Fortran used this scheme
- Data persists through life of program

■ Disadvantages:

- Cannot handle recursion or parallel execution
- Consumes resource for inactive functions

Real Solution: Run-time Stack

Calling Function

Caller

1. push new record on stack
2. copy values into arguments
3. call function

1. get result from callee

Callee

1. execute code
2. put result in record
3. pop record from stack
4. return

- **arguments, return value, and return address** are stored on the stack

- The **return address** (PC+1) MUST BE saved before transfer control !

- What instruction is used to transfer control to a function?
 - JSR/JSRR ($PC+1 \leftarrow R7$)
- What instruction is used to return from a function and How?
 - RET (JMP R7)

Run-time Stack Requirements

■ Consider what has to happen in a function call:

- Caller must pass parameters to the callee.
- Caller must transfer control to the callee.
- Caller must allocate space for the return value.
- Caller must save the return address.
- Callee requires space for local variables.
- Callee must return control to the caller.

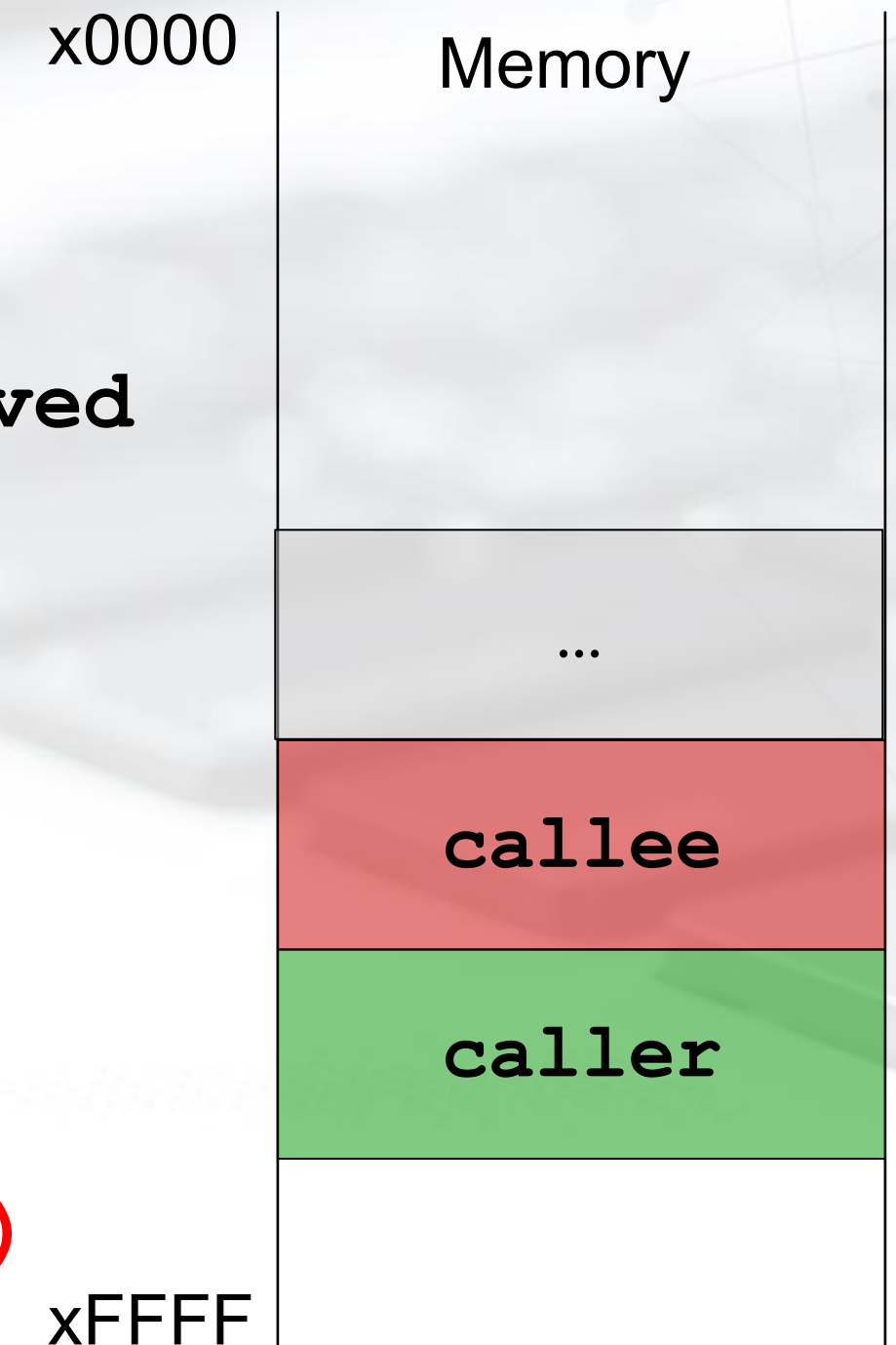
■ *Parameters, return value, return address, and locals* are stored on the stack.

■ The order above determines the responsibility and order of stack operations.

Run-time Stack

■ What is a Run-time Stack?

- First In, Last Out (FILO) data structure
- PUSH adds data, POP removes data
- Stack grows and shrinks as data is added and removed
- Stack grows downward from the end of memory space
- Function calls allocate a **stack frame**
- Return cleans up by freeing the **stack frame**
- Corresponds nicely to nested function calls
- **Stack Trace shows current execution (Java/Eclipse)**



Run-time Stack: Stack frame (or **activation record**)

■ **Definition:** A **stack frame** or **activation record** is the memory required for a function call:

- **Stack frame below** contains the function that called this function.
- **Stack frame above** contains the functions called from this function.
- **Caller** pushes parameters.
- **Callee** allocates the return value, saves the return address, allocates/frees local variables, and stores the return value.
- Most offsets are small, this explains LDR/STR implementation.
- Base register stores pointer, signed Parameters offset accesses both directions.

Calling Function



Caller

1. push new record on stack
2. copy values into arguments
3. call function

4. get result from callee

Callee

1. alloc return value in record
2. save the return address
3. alloc local variables
4. execute code
5. put result in record
6. pop record from stack
7. return

Run-time Stack: Stack frame

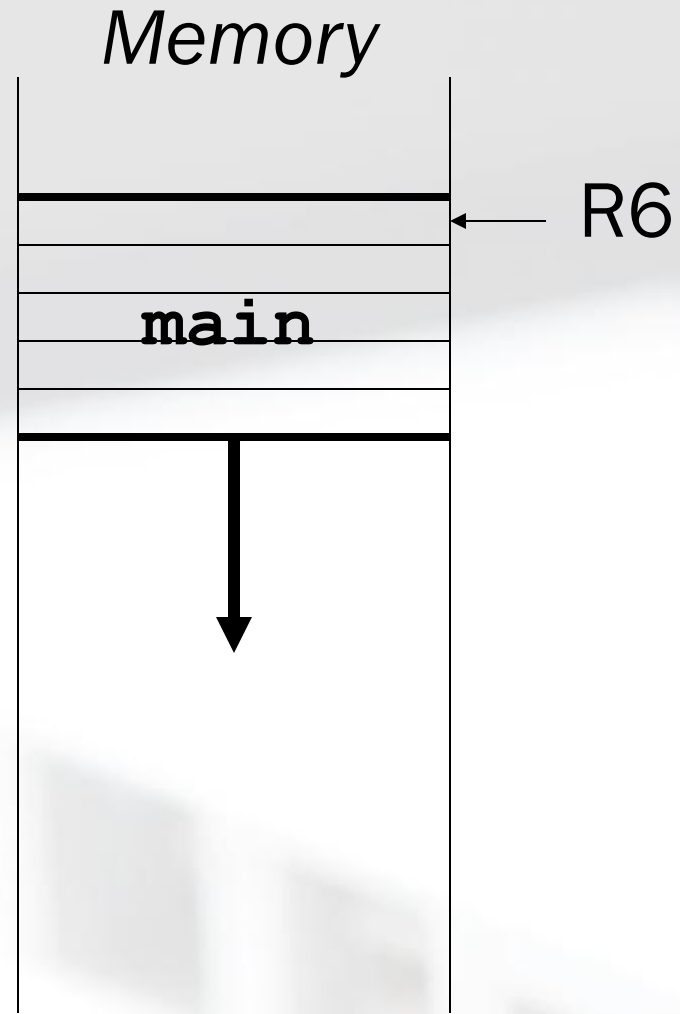
- Each function has a **memory template** where it stores its local variables, some bookkeeping information, and its parameter variables. This template is called its **stack frame** or **activation record**.
- Whenever a function is called, its **stack frame** will be allocated somewhere **in memory**.
- Because the calling pattern of functions naturally follows a stack-like pattern, this allocation and deallocation will follow the pushes and pops of a stack.

Run-Time Stack

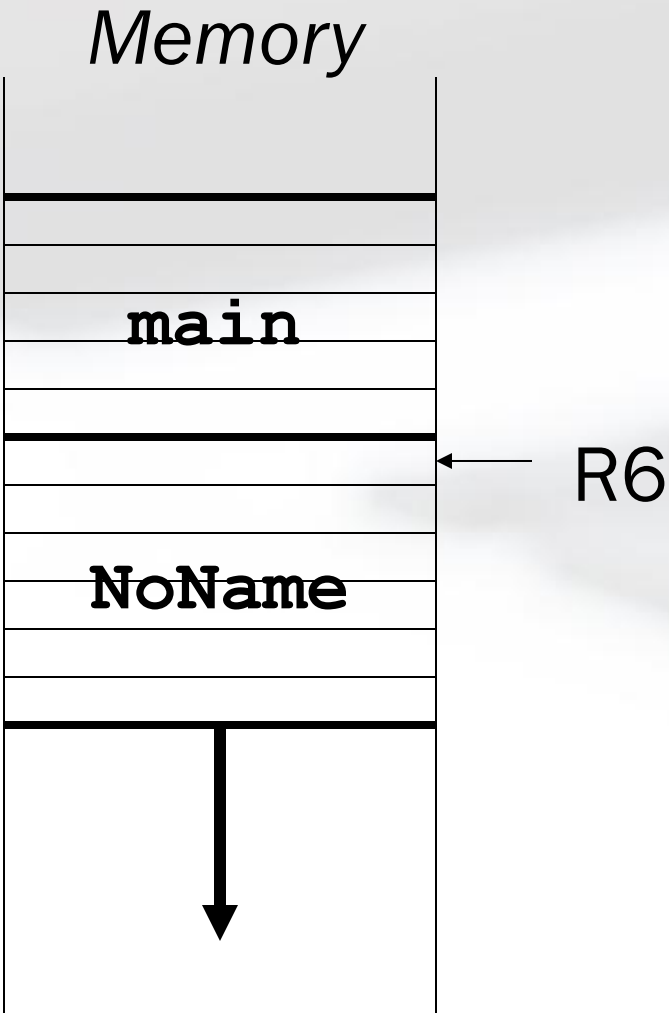
Stack pointer (R6) points to the beginning of a region of memory that stores local variables for the current function.

That region of memory is called an **activation record**

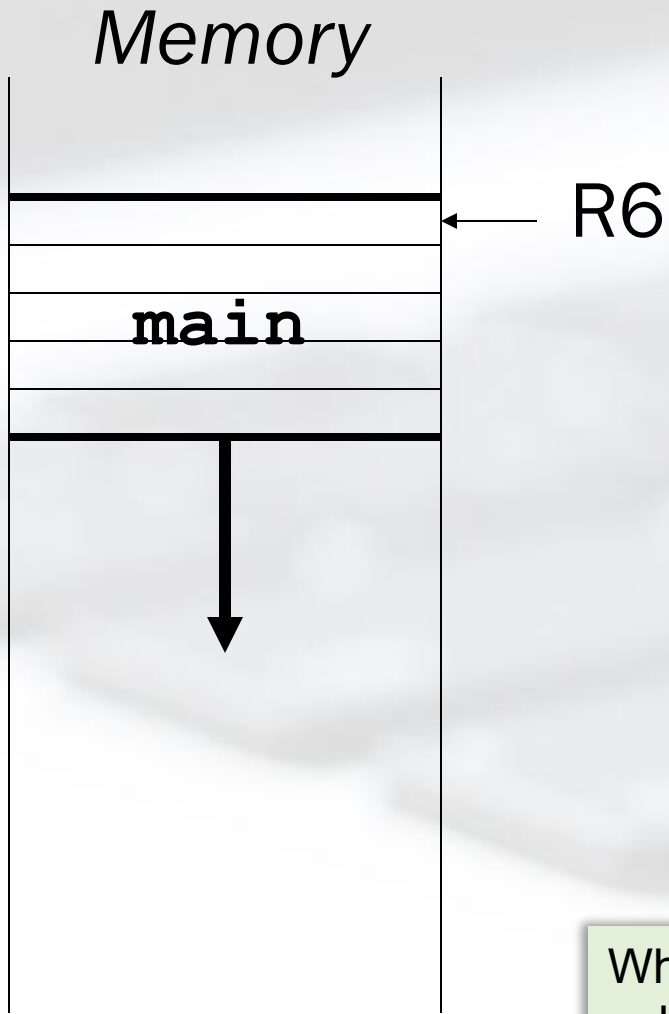
Recall that **local variables** are stored on the run-time stack



Before call



During call



After call

When a new function is called, its activation record is pushed on the stack; when it returns its activation record is popped off of the stack.

Note: Using R6 this way is incompatible with storing interrupt state on the stack.



Run-time Stack: Stack Pointers(SP/R6) and Frame Pointers(FP/R5)

■ Clearly we need a variable to store the **stack pointer (SP)**, LC3 assembly uses **R6**.

- Stack execution is ubiquitous, so hardware has a stack pointer, sometimes even instructions.
- Problem: stack pointer is difficult to use to access data, since it moves around constantly.

■ Solution: allocate another variable called a **frame pointer (FP)**, LC3 assembly uses **R5**.

- Where should frame pointer point? Our convention sets it to point to the first local variable.

Run-time Stack: frame pointer & stack pointer

- We need some easy way to access the data in each function' s stack frame and also to manage the pushing and popping of stack frames.
- For this, we will use R5 and R6.
 - R5 points to some **internal location** within the **stack frame** at the top of the stack—it may point to the base of the local variables for the currently executing function. We call it the **frame pointer (FP)**.
 - R6 always points to the very top of the **stack**. We call it the **stack pointer (SP)**.

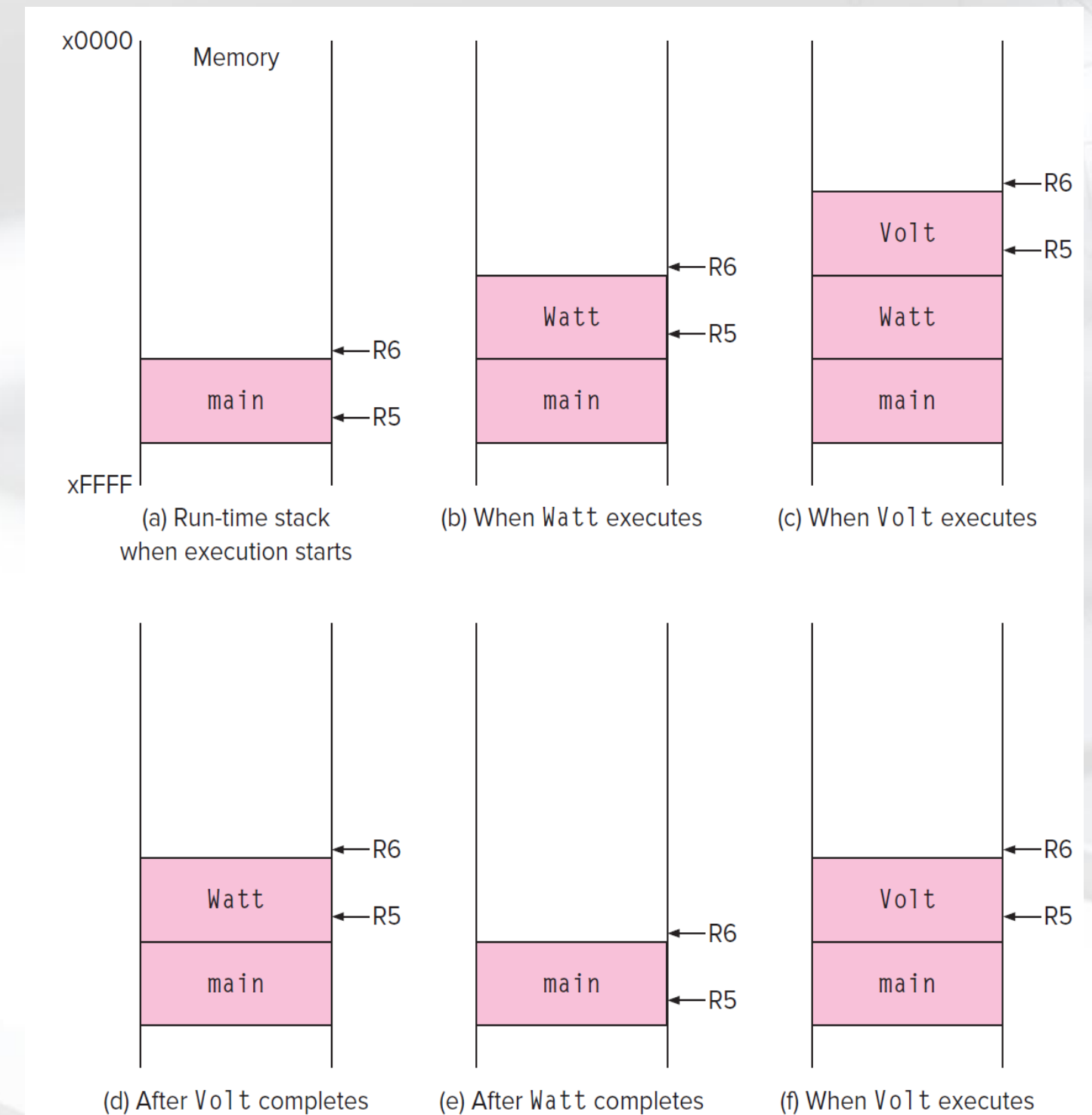


Figure 14.5 Several snapshots of the run-time stack while the program outlined in Figure 14.4 executes.

Run-time Stack: stack-like nature of function calls

```

1  int main(void)
2  {
3      int a;
4      int b;
5
6      :
7      b = Watt(a);    // main calls Watt first
8      b = Volt(a, b); // then calls Volt
9  }
10
11 int Watt(int a)
12 {
13     int w;
14
15     :
16     w = Volt(w, 10); // Watt calls Volt
17
18     return w;
19 }
20
21 int Volt(int q; int r)
22 {
23     int k;
24     int m;
25
26     :
27     return k;
28 }

```

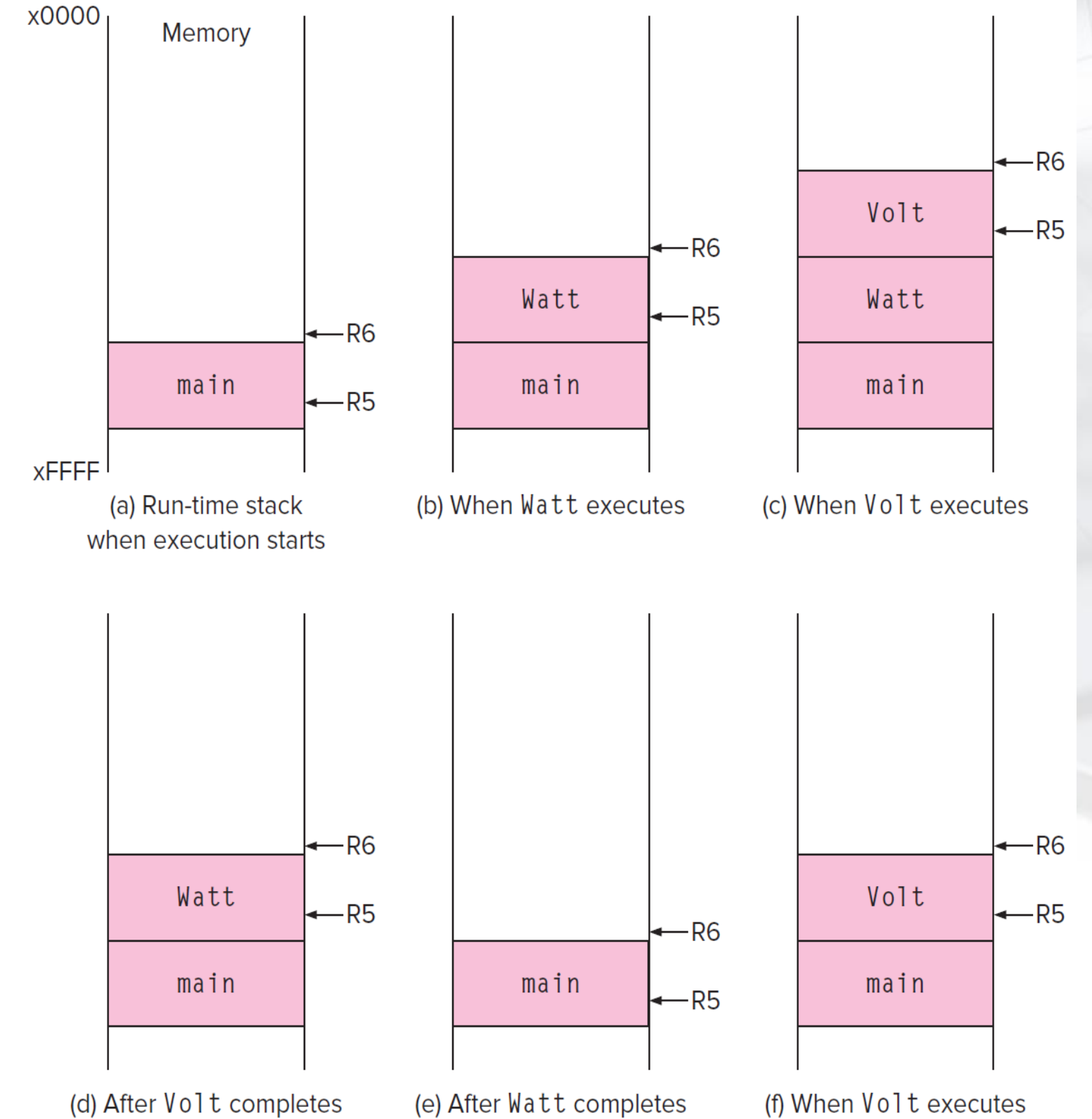


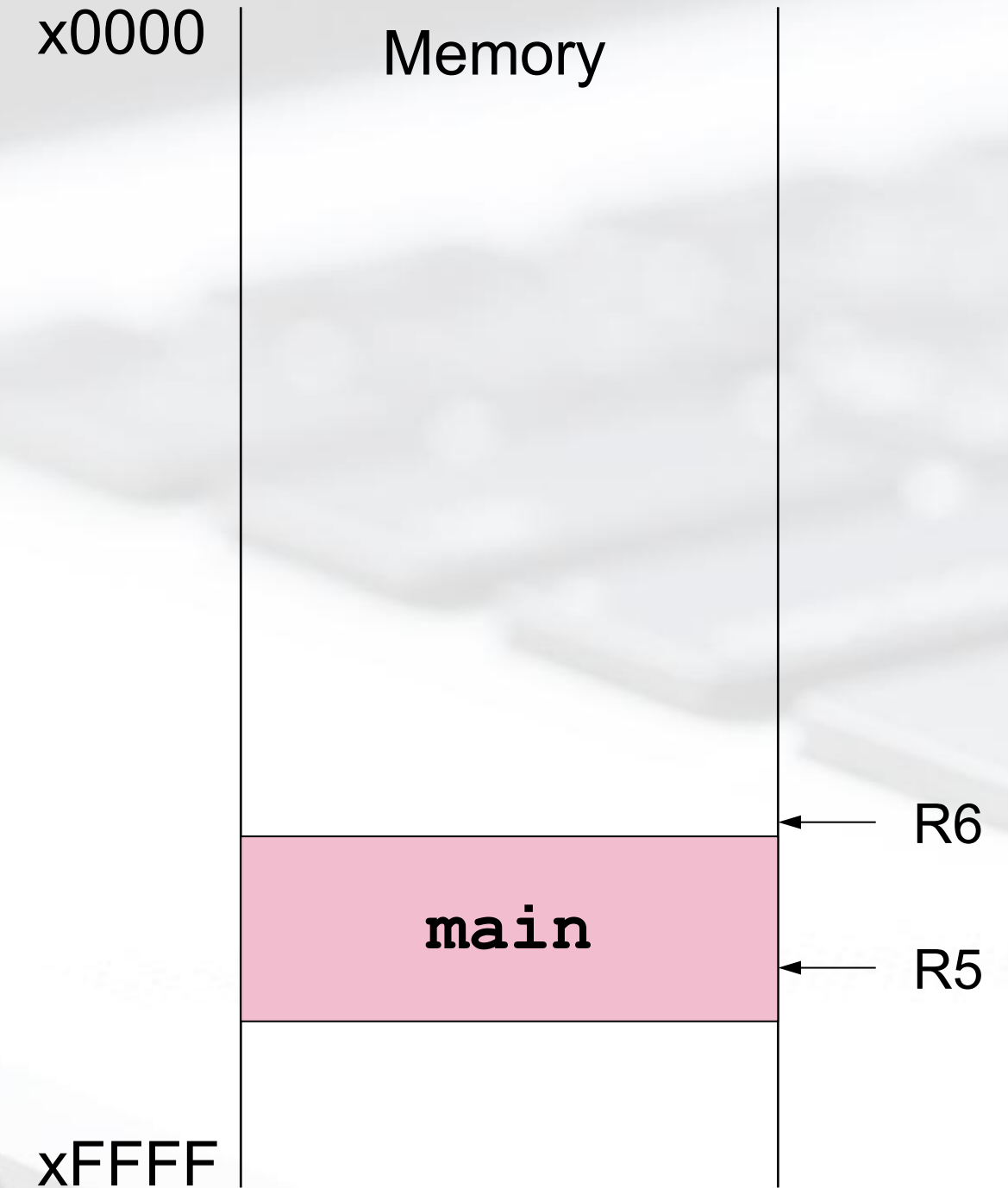
Figure 14.5 Several snapshots of the run-time stack while the program outlined in Figure 14.4 executes.

Run-Time Stack: Example

```
1  int main(void)
2  {
3      int a;
4      int b;
5      →
6      b = Watt(a);    // main calls Watt first
7      b = Volt(a, b); // then calls Volt
8  }
9
10 int Watt(int a)
11 {
12     int w;
13
14     w = Volt(w, 10); // Watt calls Volt
15
16     return w;
17 }
18
19 int Volt(int q; int r)
20 {
21     int k;
22     int m;
23
24     return k;
25 }
```

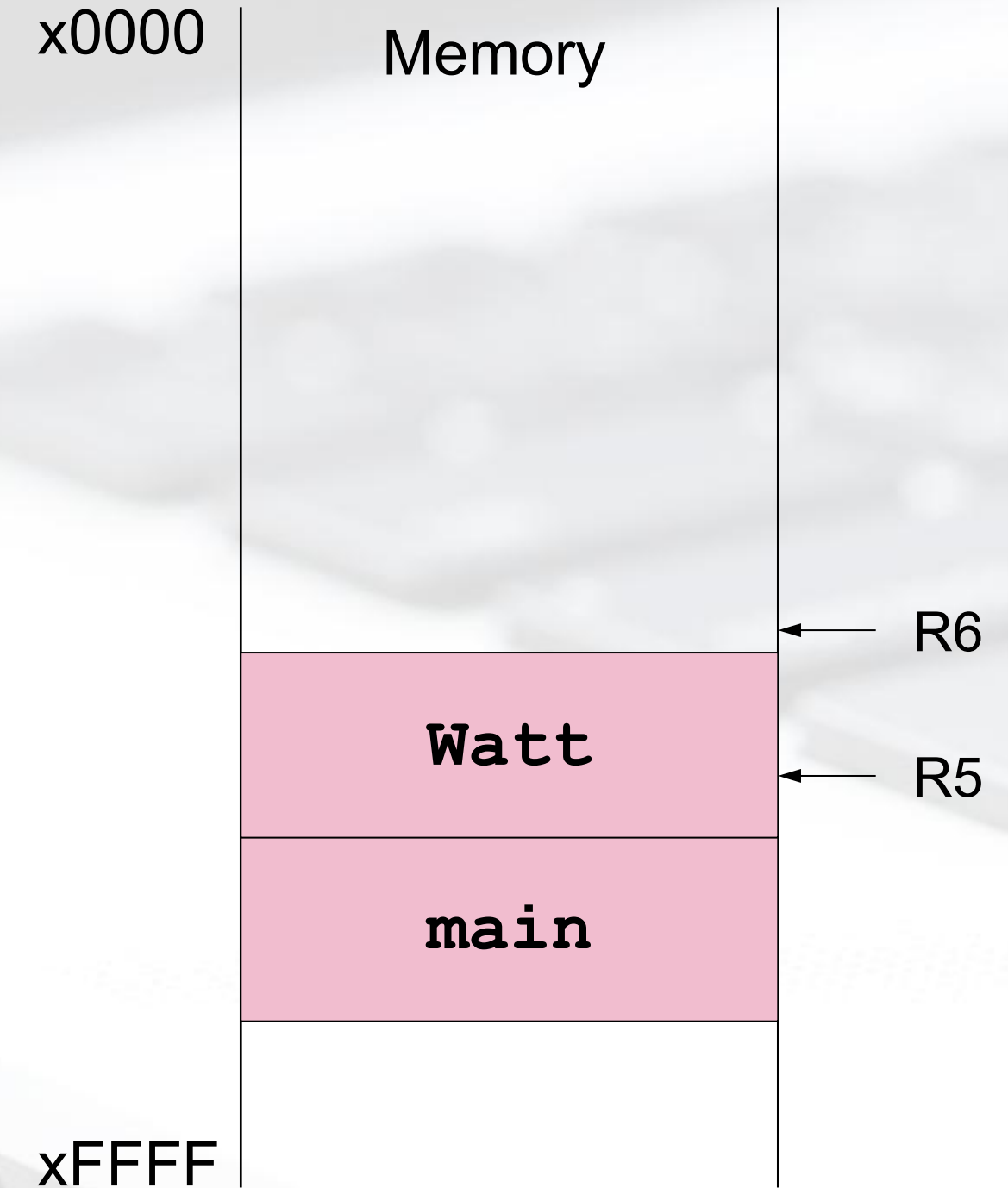
Run-time stack when execution starts

```
1  int main(void)
2  {
3      int a;
4      int b;
5
6      b = Watt(a);    // main calls Watt first
7      b = Volt(a, b); // then calls Volt
8  }
9
10 int Watt(int a)
11 {
12     int w;
13
14     w = Volt(w, 10); // Watt calls Volt
15
16     return w;
17 }
18
19 int Volt(int q; int r)
20 {
21     int k;
22     int m;
23
24     return k;
25 }
```



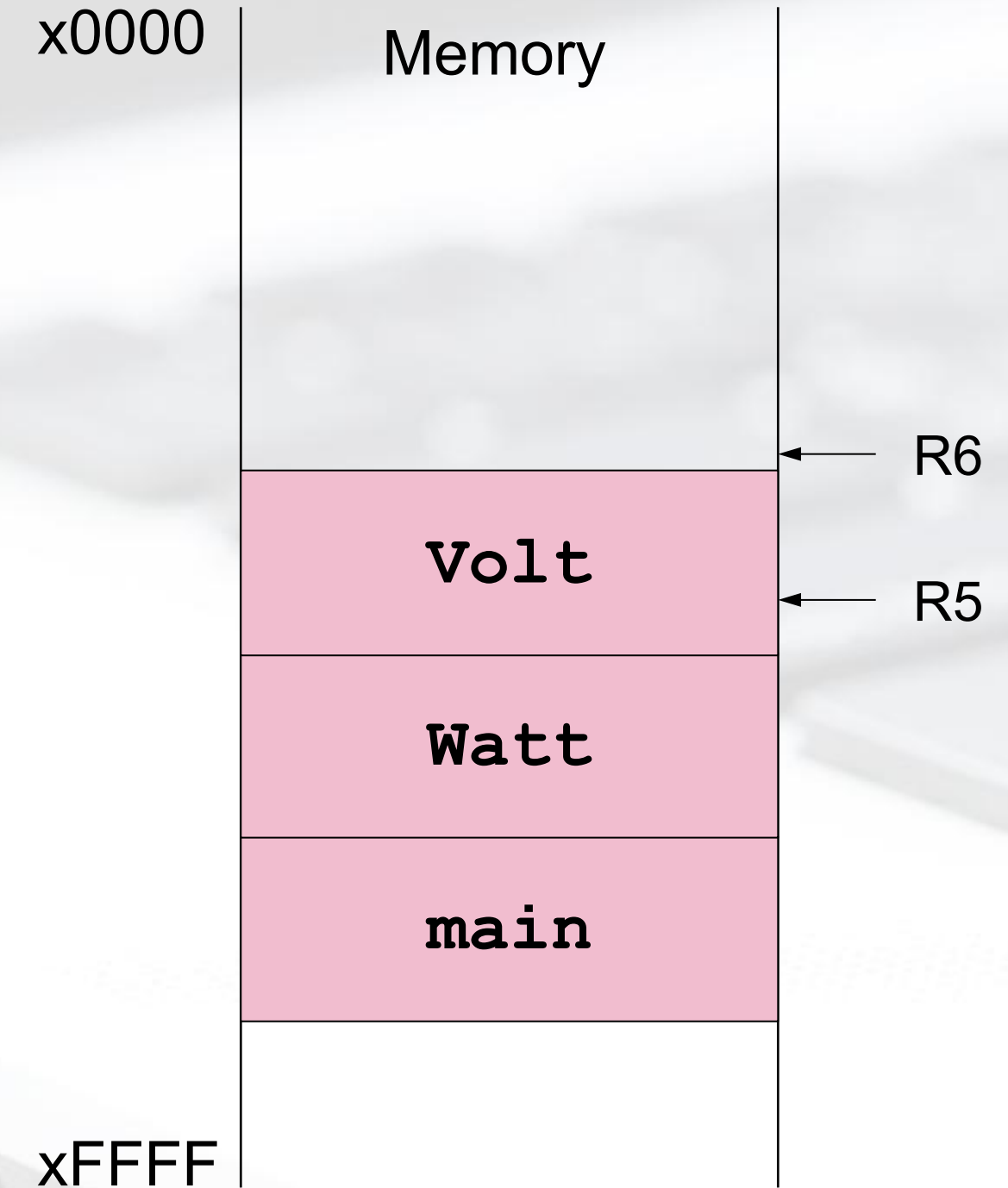
Run-time stack when Watt executes

```
1  int main(void)
2  {
3      int a;
4      int b;
5
6      b = Watt(a);    // main calls Watt first
7      b = Volt(a, b); // then calls Volt
8  }
9
10 int Watt(int a)
11 {
12     int w;
13
14     w = Volt(w, 10); // Watt calls Volt
15
16     return w;
17 }
18
19 int Volt(int q; int r)
20 {
21     int k;
22     int m;
23
24     return k;
25 }
```



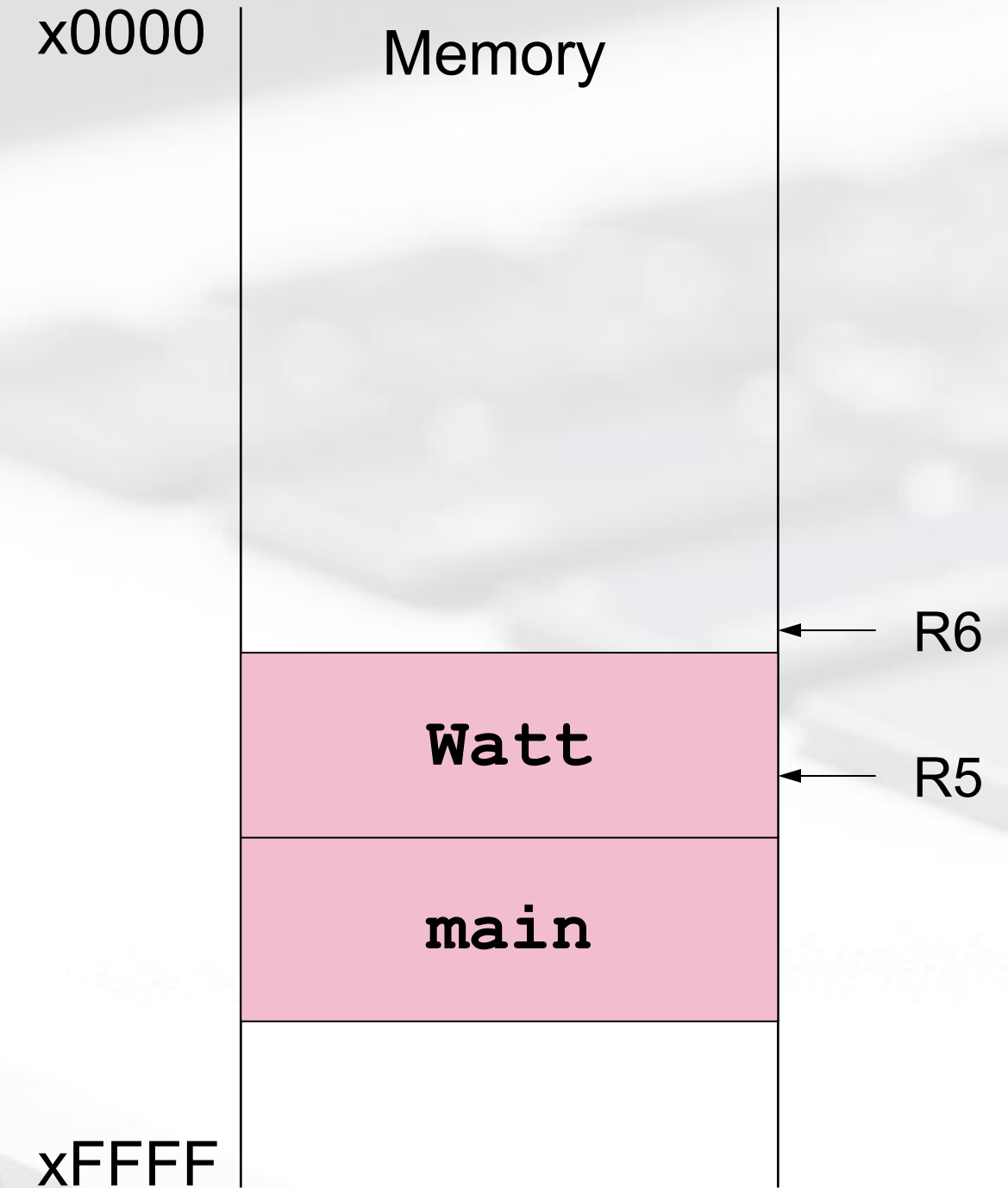
Run-time stack when Volt executes

```
1  int main(void)
2  {
3      int a;
4      int b;
5
6      b = Watt(a);    // main calls Watt first
7      b = Volt(a, b); // then calls Volt
8  }
9
10 int Watt(int a)
11 {
12     int w;
13
14     w = Volt(w, 10); // Watt calls Volt
15
16     return w;
17 }
18
19 int Volt(int q; int r)
20 {
21     int k;
22     int m;
23
24     return k;
25 }
```



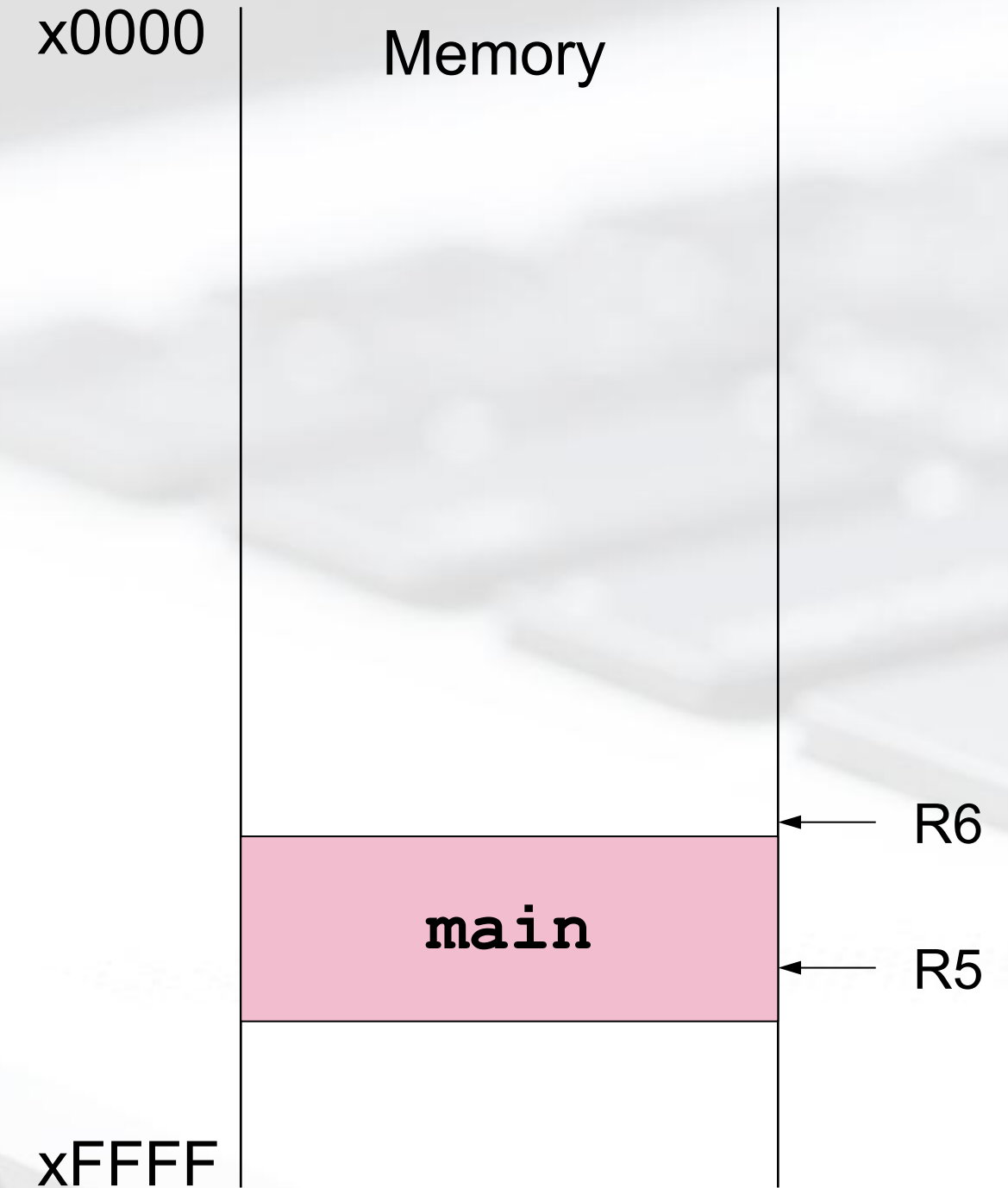
Run-time stack after Volt completes

```
1  int main(void)
2  {
3      int a;
4      int b;
5
6      b = Watt(a);    // main calls Watt first
7      b = Volt(a, b); // then calls Volt
8  }
9
10 int Watt(int a)
11 {
12     int w;
13
14     w = Volt(w, 10); // Watt calls Volt
15
16     return w;
17 }
18
19 int Volt(int q; int r)
20 {
21     int k;
22     int m;
23
24     return k;
25 }
```



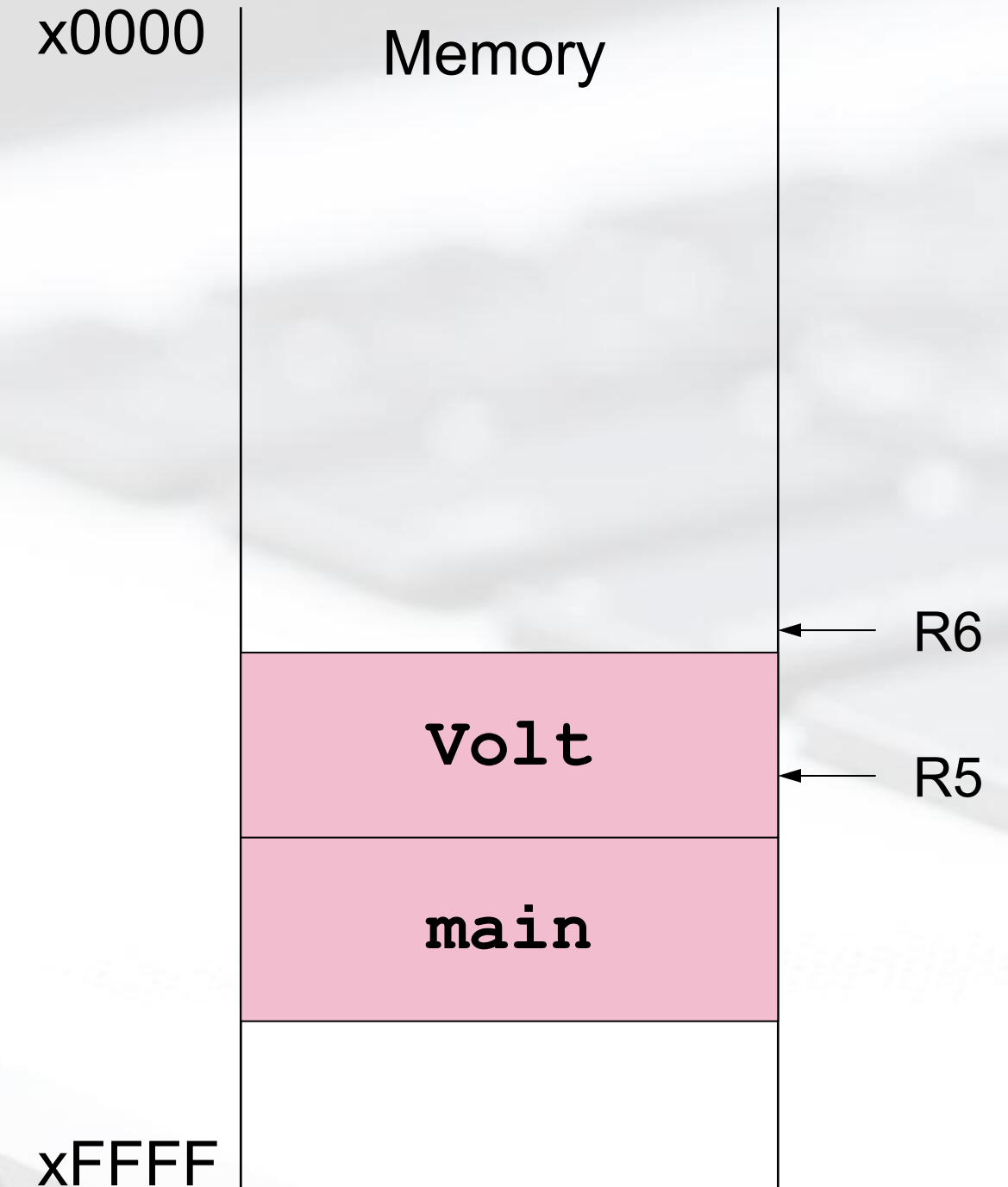
Run-time stack after Watt completes

```
1  int main(void)
2  {
3      int a;
4      int b;
5
6      b = Watt(a);    // main calls Watt first
7      b = Volt(a, b); // then calls Volt
8  }
9
10 int Watt(int a)
11 {
12     int w;
13
14     w = Volt(w, 10); // Watt calls Volt
15
16     return w;
17 }
18
19 int Volt(int q; int r)
20 {
21     int k;
22     int m;
23
24     return k;
25 }
```



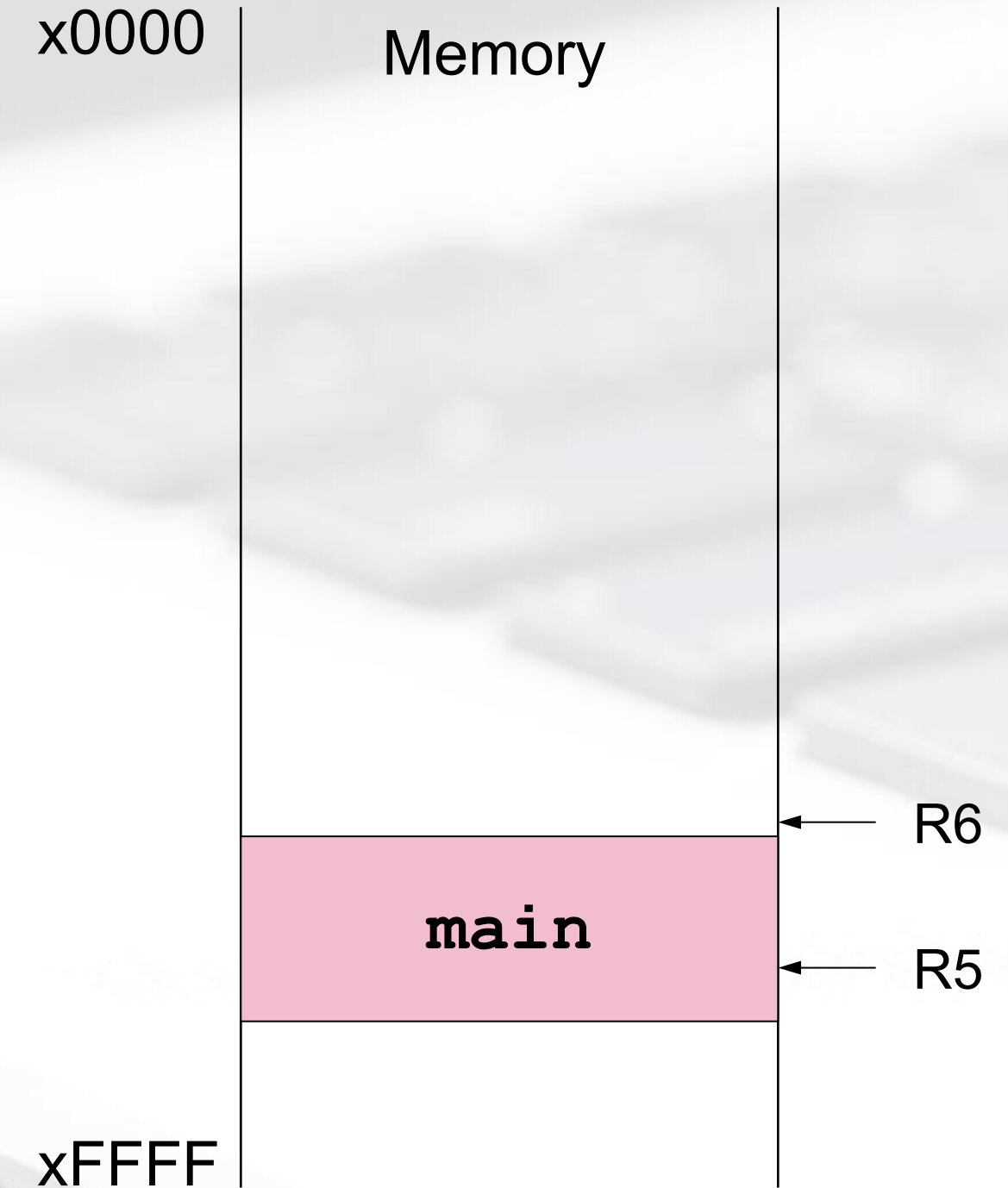
Run-time stack when Volt executes

```
1  int main(void)
2  {
3      int a;
4      int b;
5
6      b = Watt(a);    // main calls Watt first
7      b = Volt(a, b); // then calls Volt
8  }
9
10 int Watt(int a)
11 {
12     int w;
13
14     w = Volt(w, 10); // Watt calls Volt
15
16     return w;
17 }
18
19 int Volt(int q; int r)
20 {
21     int k;
22     int m;
23
24     return k;
25 }
```



Run-time stack after Volt completes

```
1  int main(void)
2  {
3      int a;
4      int b;
5
6      b = Watt(a);    // main calls Watt first
7      b = Volt(a, b); // then calls Volt
8  }
9
10 int Watt(int a)
11 {
12     int w;
13
14     w = Volt(w, 10); // Watt calls Volt
15
16     return w;
17 }
18
19 int Volt(int q; int r)
20 {
21     int k;
22     int m;
23
24     return k;
25 }
```



Run-time Stack: frame pointer

- In the previous solutions, the compiler allocated parameters and locals in fixed memory locations.
- Using a Run-time Stack means parameters and locals are constantly moving around.
- **The frame pointer** solves this problem by using fixed offsets instead of addresses.
- The compiler can generate code using offsets, without knowing where the stack frame will reside.
- **Frame pointer** needs to be saved and restored around function calls. How about the stack pointer?

Implementing Functions in C Using a Run-Time Stack

■ Activation record

- information about each function, including arguments and local variables
- stored on run-time stack

Caller

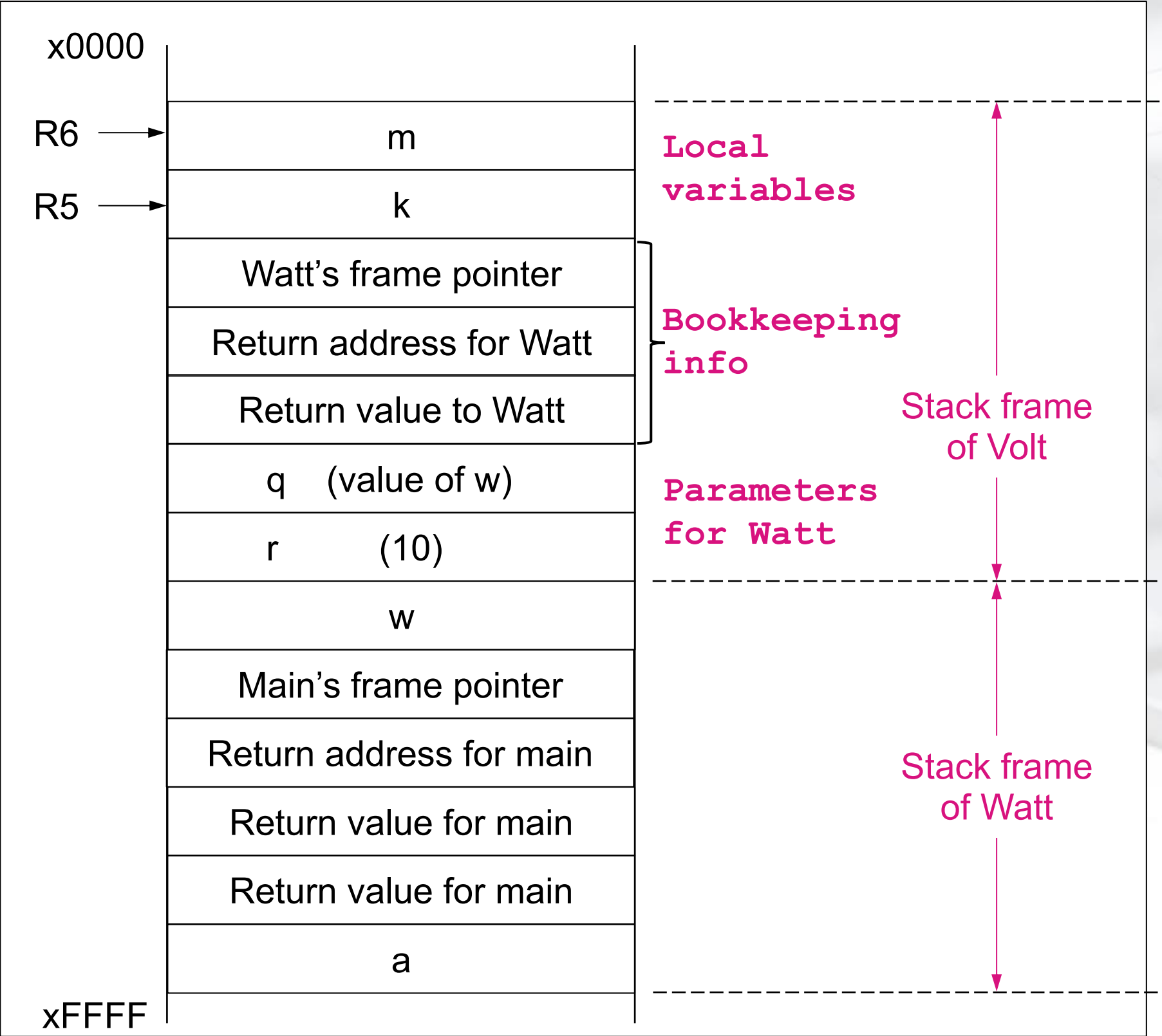
1. push new record on stack
2. copy values into arguments
3. call function

4. get result from callee

Callee

1. alloc return value in record
2. save the return address
3. alloc local variables
4. execute code
5. put result in record
6. pop record from stack
7. return

```
11 int Watt(int a)
12 {
13     int w;
14
15     :
16     w = Volt(w, 10);    // Watt calls Volt
17
18     return w;
19 }
20
21 int Volt(int q; int r)
22 {
23     int k;
24     int m;
25
26     :
27     return k;
28 }
```



Run-time Stack: Stack frame

■ Consider what has to happen in a function call:

- Caller must pass parameters to the callee.
- Caller must transfer control to the callee.
- Caller need to allocate space for the return value.
- Caller need to save the return address.
- Callee requires space for local variables.
- Callee must return control to the caller.
- Callee need to save the **frame pointer of the caller**

■ So, parameters, return value, return address, frame pointer, and local variables are stored on the stack.

Function call in C

■ Four steps of the C-runtime stack protocol

1. The **caller** copies arguments for the **callee** onto the run-time stack and passes control to the callee
2. The **callee** function pushes space for **local variables** and other info. onto the run-time stack, creating its **stack frame** on top of the stack
3. The **callee** executes the task
4. Once it is ready to return, the **callee** removes, or pops, its stack frame off the run-time stack and returns the **return value** and control to the caller



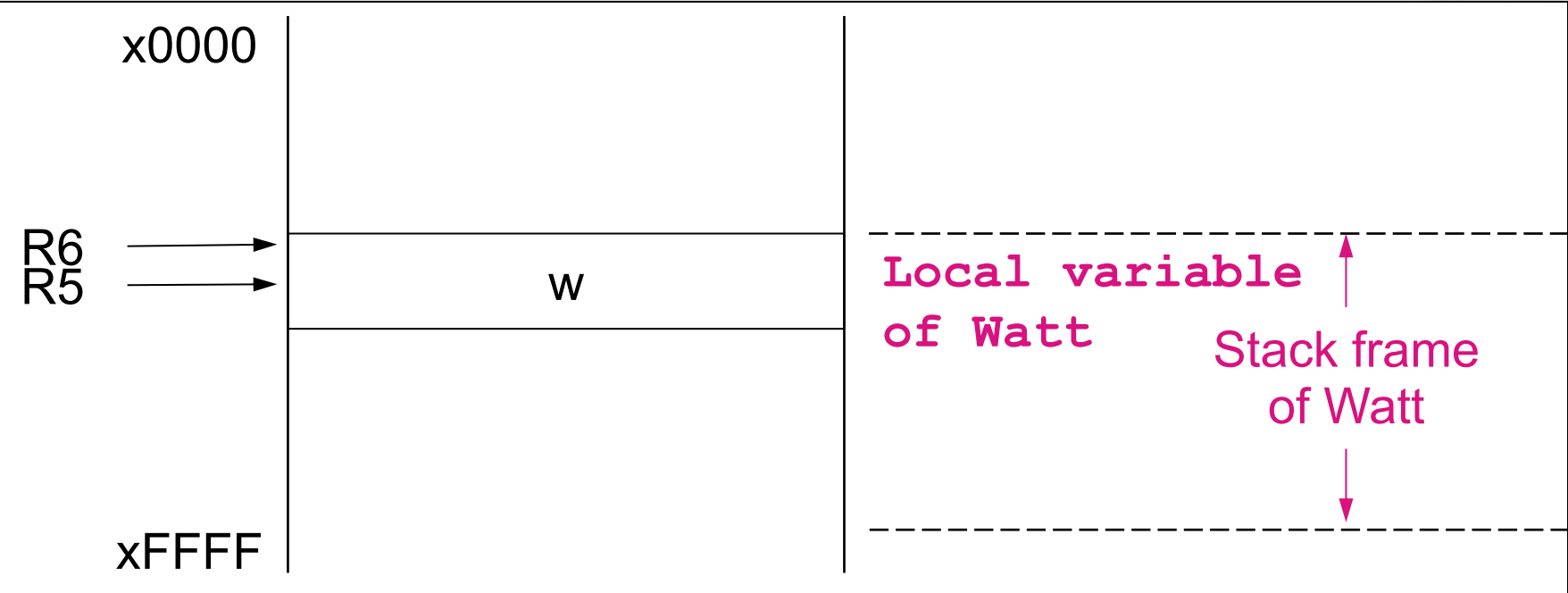
■ Four steps of the C-runtime stack protocol

- 1. The **caller** copies arguments for the **callee** onto the run-time stack and passes control to the callee

```
10 int Watt(int a)
11 {
12     int w;
13
14     w = Volt(w, 10);
15
16     return w;
17 }
18
19 int Volt(int q; int
    r)
20 {
21     int k;
22     int m;
23
24     return k;
25 }
```

```
AND R0,R0,#0 ; R0 <- 0
ADD R0,R0,#10 ; R0 <- 10
ADD R6,R6,#-1 ;
STR R0,R6,#0 ; Push 10 onto stack

LDR R0,R5,#0 ; Load w
ADD R6,R6,#-1 ;
STR R0,R6,#0 ; Push w
JSR Volt
```





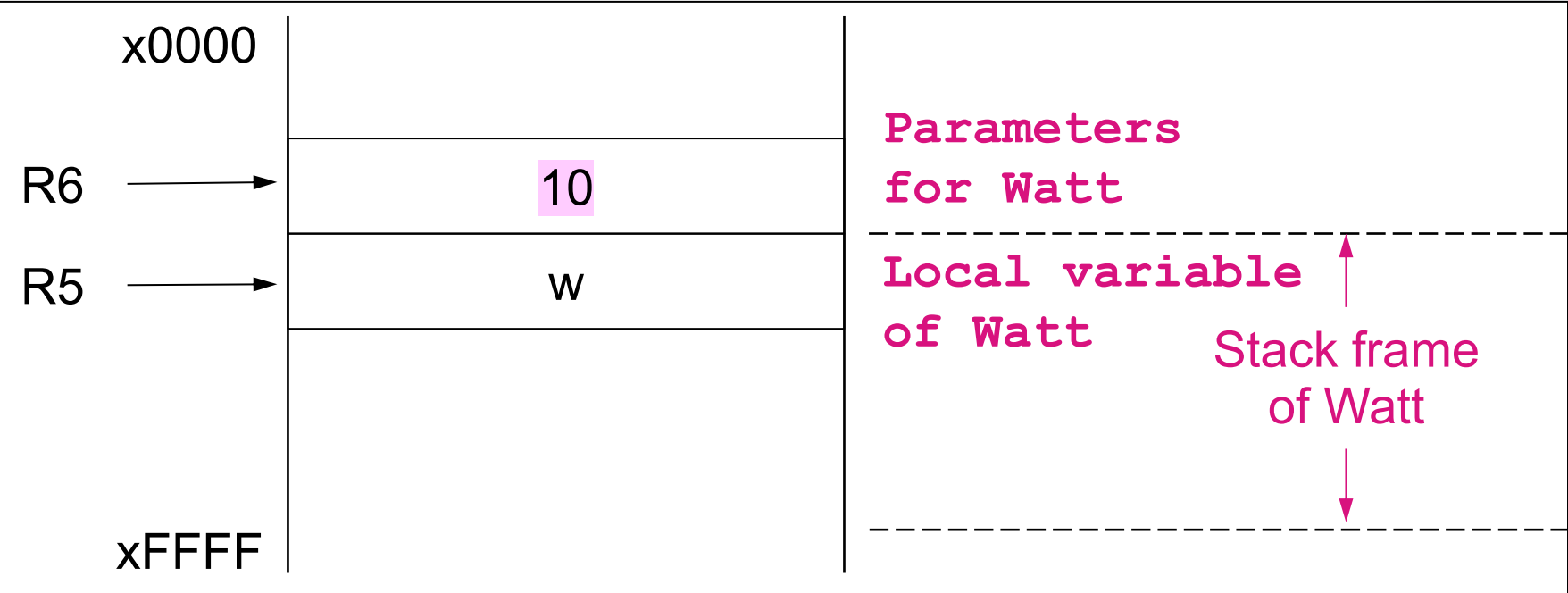
■ Four steps of the C-runtime stack protocol

- 1. The **caller** copies arguments for the **callee** onto the run-time stack and passes control to the callee

```
10 int Watt(int a)
11 {
12     int w;
13
14     w = Volt(w, 10);
15
16     return w;
17 }
18
19 int Volt(int q; int
20         r)
21 {
22     int k;
23     int m;
24     return k;
25 }
```

```
AND R0,R0,#0 ; R0 <- 0
ADD R0,R0,#10 ; R0 <- 10
ADD R6,R6,#-1 ;
STR R0,R6,#0 ; Push 10 onto stack

LDR R0,R5,#0 ; Load w
ADD R6,R6,#-1 ;
STR R0,R6,#0 ; Push w
JSR Volt
```





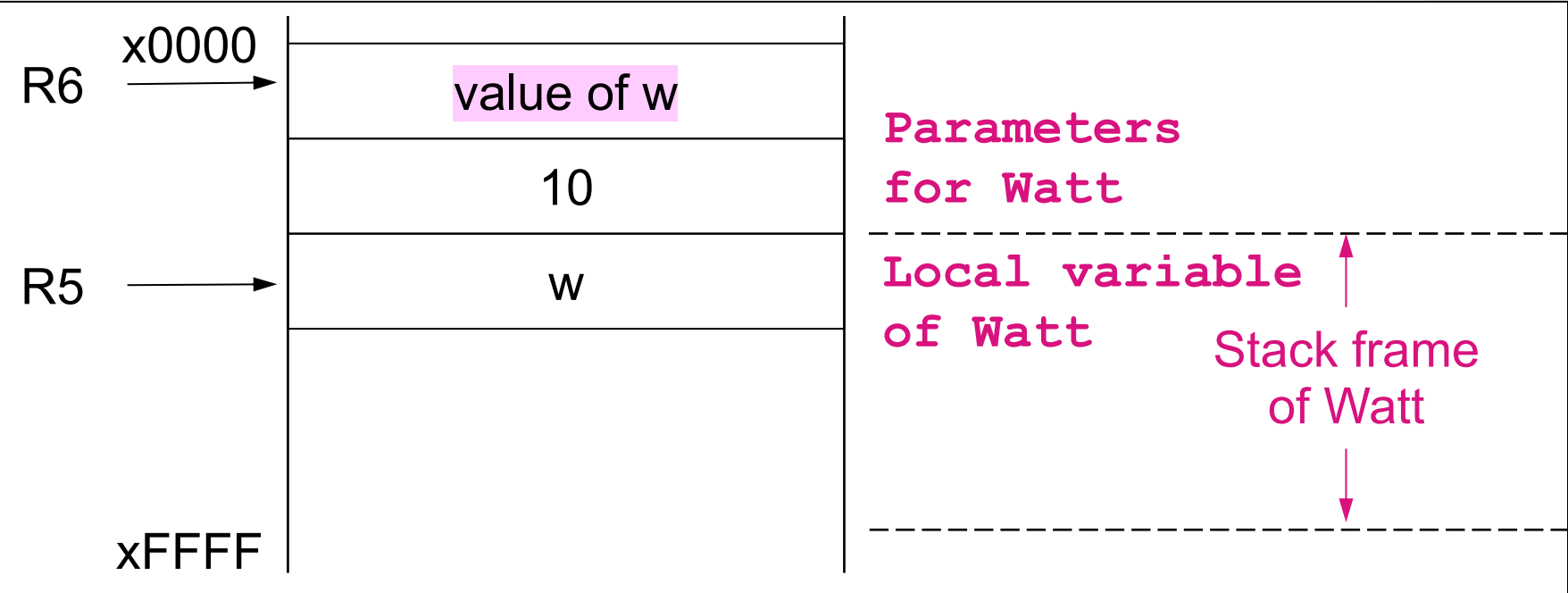
■ Four steps of the C-runtime stack protocol

- 1. The **caller** copies arguments for the **callee** onto the run-time stack and passes control to the callee

```
10 int Watt(int a)
11 {
12     int w;
13
14     w = Volt(w, 10);
15
16     return w;
17 }
18
19 int Volt(int q; int
20         r)
21 {
22     int k;
23     int m;
24     return k;
25 }
```

```
AND R0,R0,#0 ; R0 <- 0
ADD R0,R0,#10 ; R0 <- 10
ADD R6,R6,#-1 ;
STR R0,R6,#0 ; Push 10 onto stack

LDR R0,R5,#0 ; Load w
ADD R6,R6,#-1 ;
STR R0,R6,#0 ; Push w
JSR Volt
```





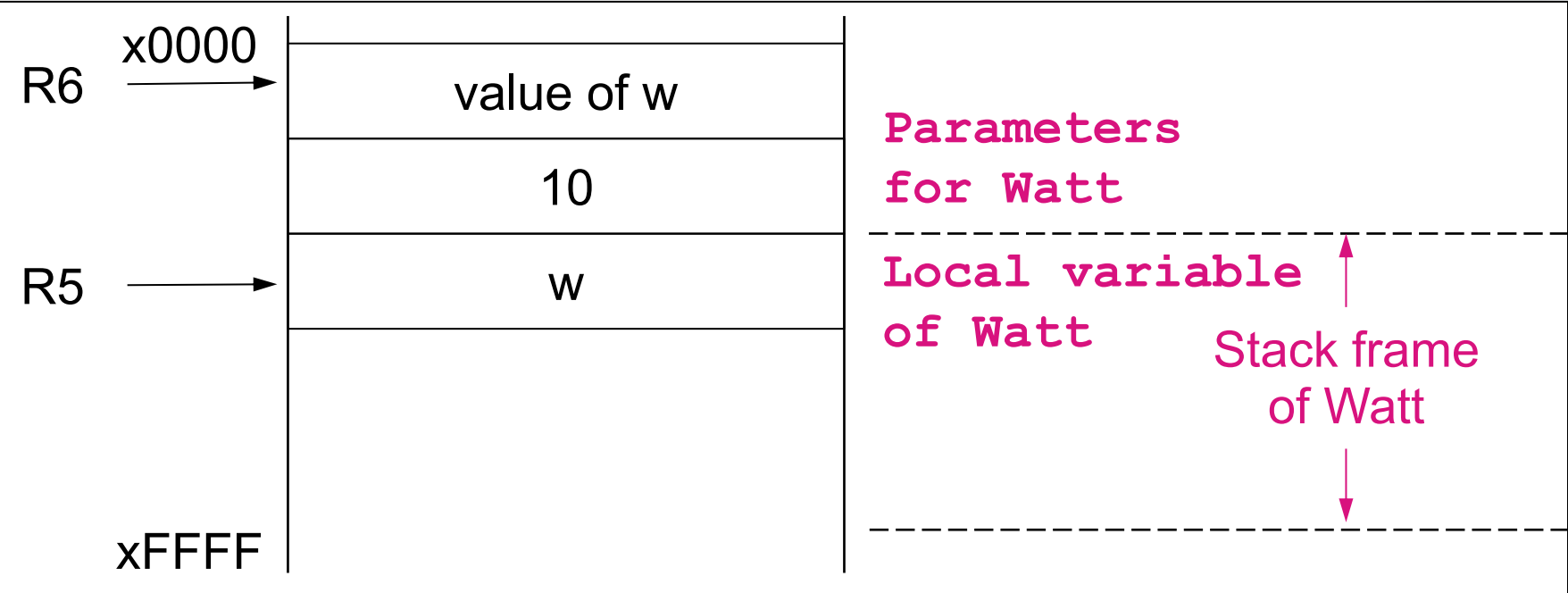
■ Four steps of the C-runtime stack protocol

- 1. The **caller** copies arguments for the **callee** onto the run-time stack and passes control to the callee

```
10 int Watt(int a)
11 {
12     int w;
13
14     w = Volt(w, 10);
15
16     return w;
17 }
18
19 int Volt(int q; int
    r)
20 {
21     int k;
22     int m;
23
24     return k;
25 }
```

```
AND R0,R0,#0 ; R0 <- 0
ADD R0,R0,#10 ; R0 <- 10
ADD R6,R6,#-1 ;
STR R0,R6,#0 ; Push 10 onto stack

LDR R0,R5,#0 ; Load w
ADD R6,R6,#-1 ;
STR R0,R6,#0 ; Push w
JSR Volt
```





■ Four steps of the C-runtime stack protocol

- 2. The **callee** function pushes space for **local variables** and other info. onto the run-time stack, creating its **stack frame** on top of the stack

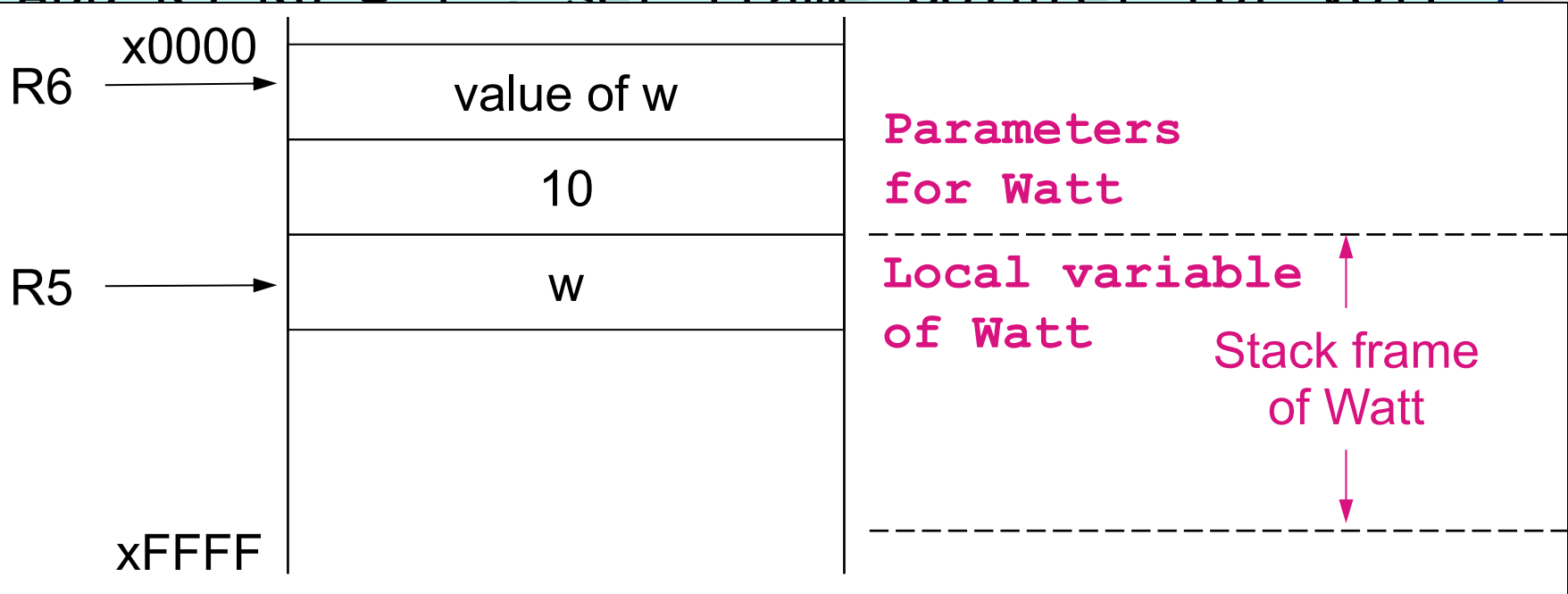
```
10 int Watt(int a)
11 {
12     int w;
13
14     w = Volt(w, 10);
15
16     return w;
17 }
18
19 int Volt(int q, int r)
20 {
21     int k;
22     int m;
23
24     return k;
25 }
```

```
Volt:
    ADD R6,R6,#-1 ; alloc. spot for the
                        ; return value

    ADD R6,R6,#-1 ;
    STR R7,R6,#0  ; push R7 (return addr.)

    ADD R6,R6,#-1
    STR R5,R6,#0  ; push R5
                        ; (caller's frame pointer)

    ADD R5,R6,#-1 ; set frame pointer for Volt
```





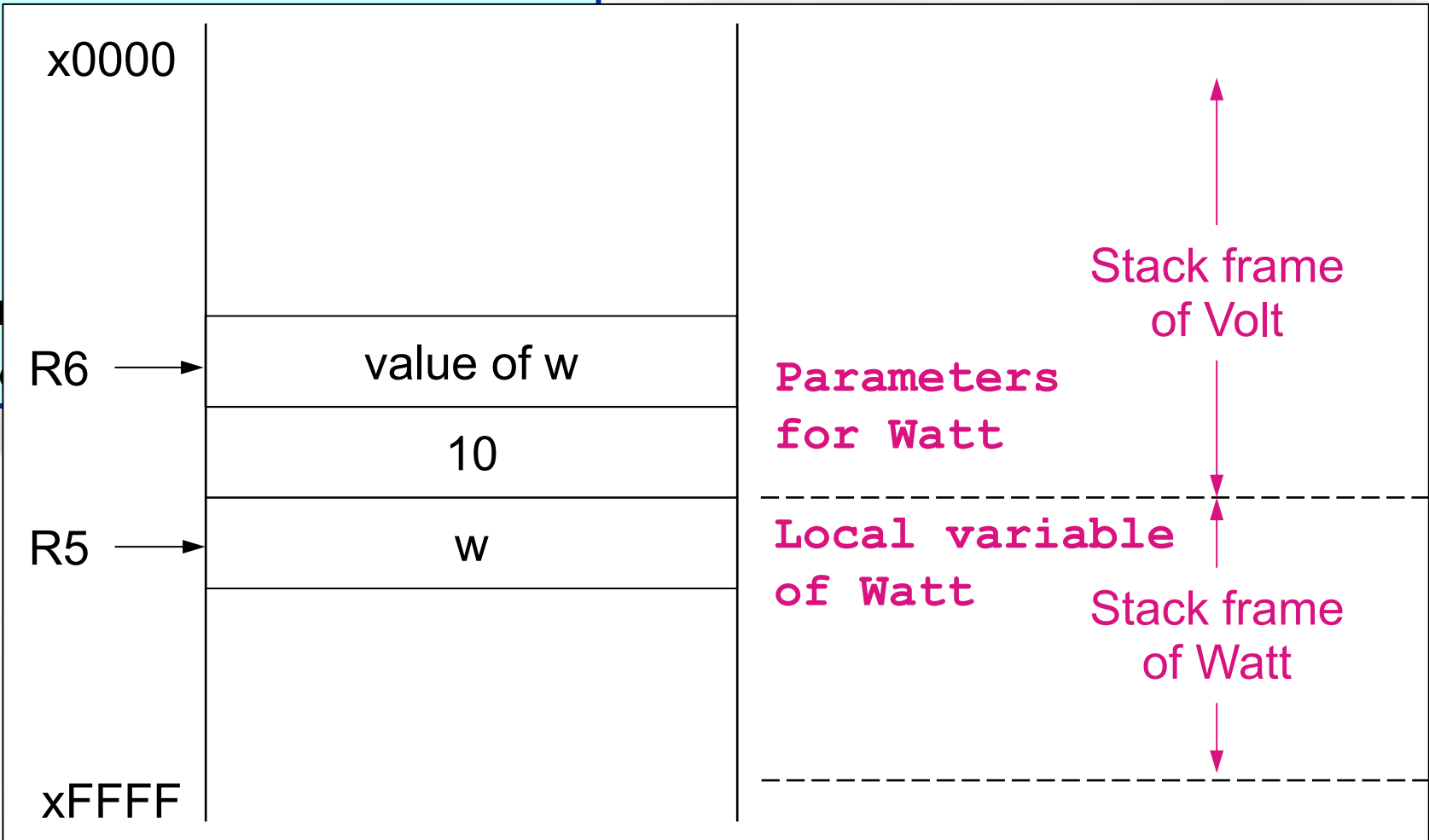
■ Four steps of the C-runtime stack protocol

- 2. The **callee** function pushes space for **local variables** and other info. onto the run-time stack, creating its **stack frame** on top of the stack

```
Volt:
  ADD R6,R6,#-1 ; alloc. spot for the
                  ; return value
  ADD R6,R6,#-1 ;
  STR R7,R6,#0  ; push R7 (return addr.)
```

```
  ADD R6,R6,#-1
  STR R5,R6,#0  ; push R5
                  ; (caller's
  ADD R5,R6,#-1 ; set frame
  ADD R6,R6,#-2 ; alloc. me
                  ; Volt's lo
```

```
18
19 int Volt(int q, int r)
20 {
21     int k;
22     int m;
23
24     return k;
25 }
```





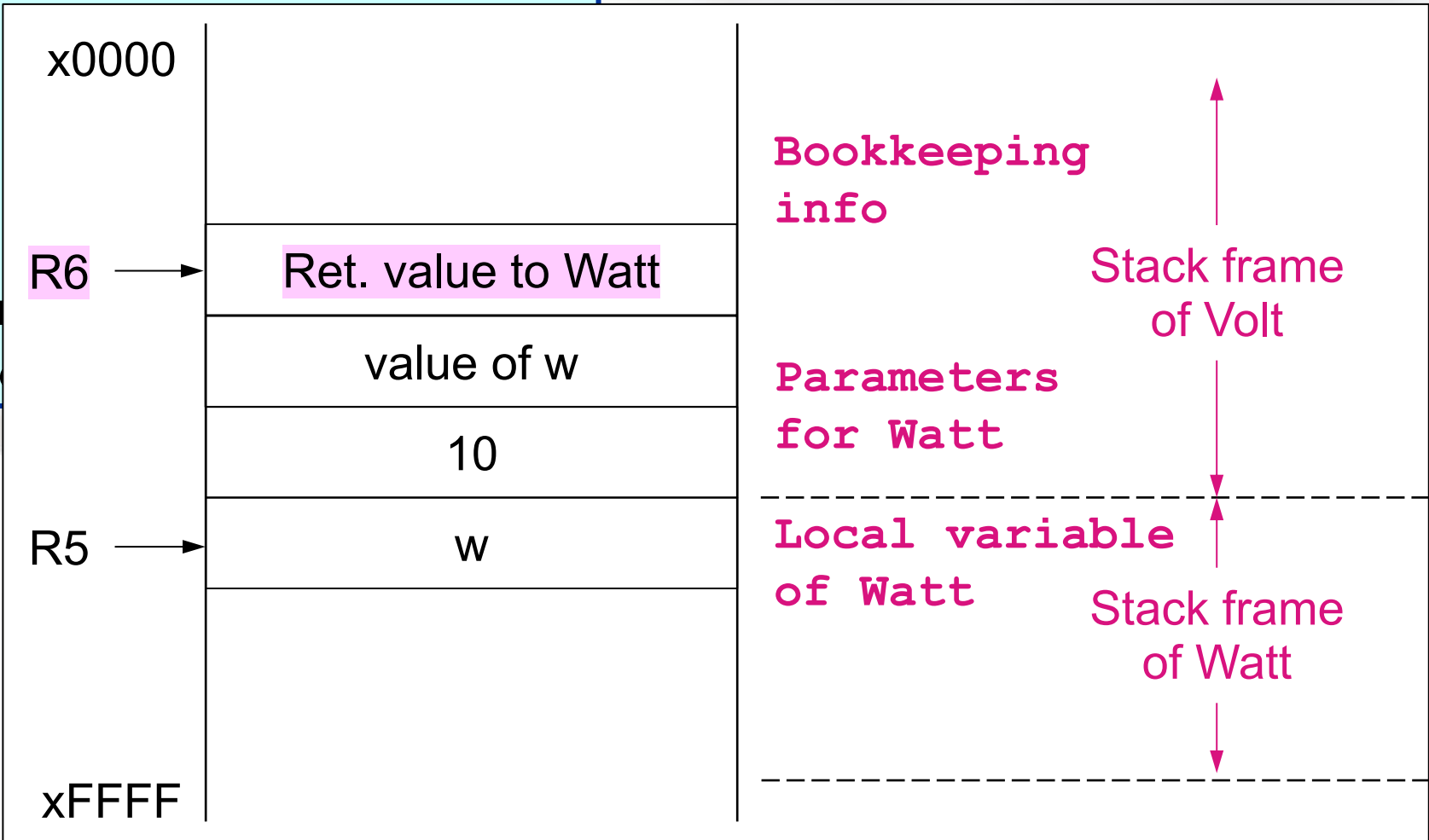
■ Four steps of the C-runtime stack protocol

- 2. The **callee** function pushes space for **local variables** and other info. onto the run-time stack, creating its **stack frame** on top of the stack

```
Volt:
  ADD R6,R6,#-1 ; alloc. spot for the
                  ; return value
  ADD R6,R6,#-1 ;
  STR R7,R6,#0  ; push R7 (return addr.)
```

```
  ADD R6,R6,#-1
  STR R5,R6,#0  ; push R5
                  ; (caller's
  ADD R5,R6,#-1 ; set frame
  ADD R6,R6,#-2 ; alloc. me
                  ; Volt's lo
```

```
18
19 int Volt(int q, int r)
20 {
21     int k;
22     int m;
23
24     return k;
25 }
```





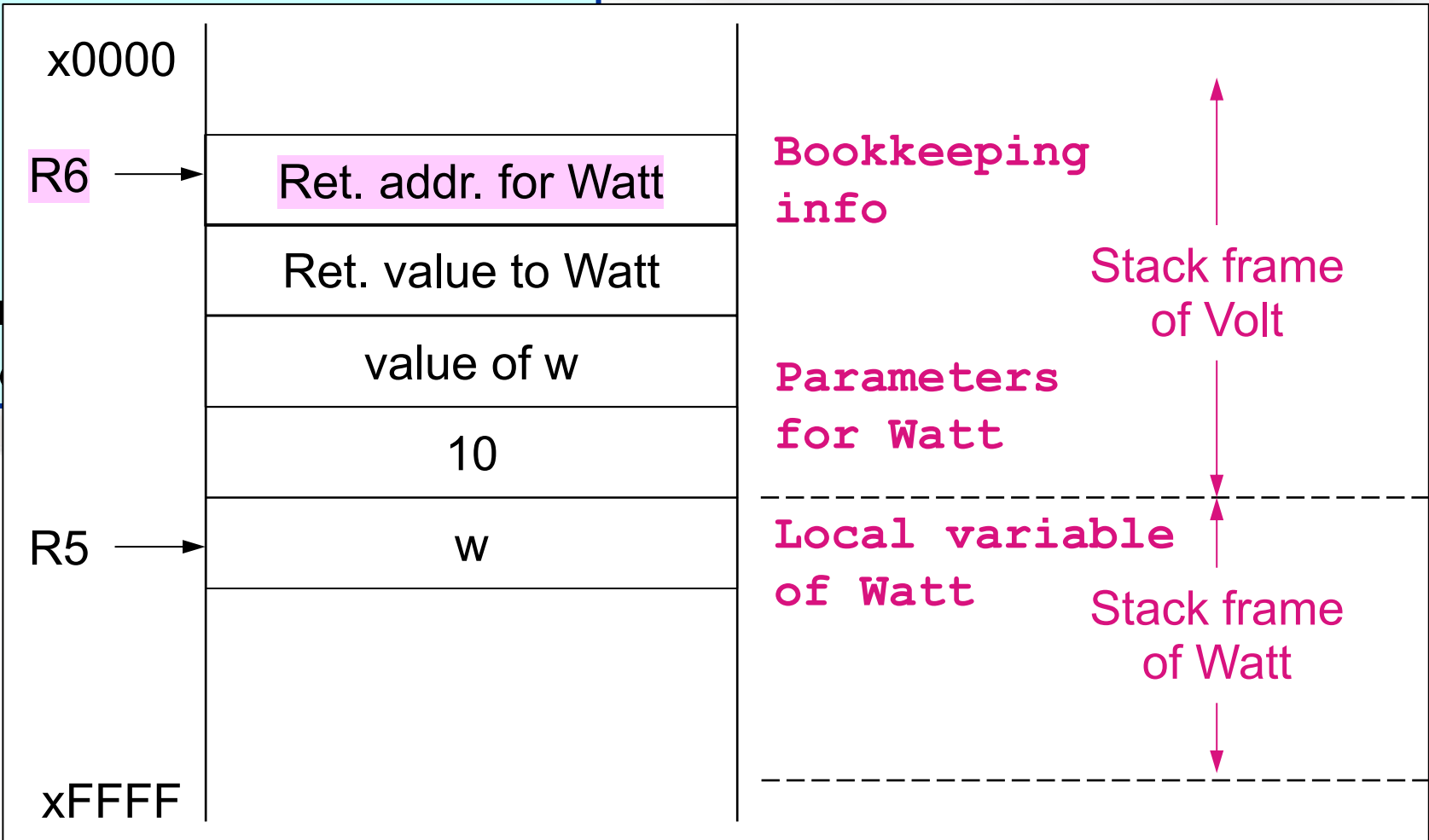
■ Four steps of the C-runtime stack protocol

- 2. The **callee** function pushes space for **local variables** and other info. onto the run-time stack, creating its **stack frame** on top of the stack

```
Volt:
  ADD R6,R6,#-1 ; alloc. spot for the
                  ; return value
  ADD R6,R6,#-1 ;
  STR R7,R6,#0  ; push R7 (return addr.)
```

```
ADD R6,R6,#-1
STR R5,R6,#0 ; push R5
              ; (caller's
ADD R5,R6,#-1 ; set frame
ADD R6,R6,#-2 ; alloc. me
              ; Volt's lo
```

```
18
19 int Volt(int q, int r)
20 {
21     int k;
22     int m;
23
24     return k;
25 }
```





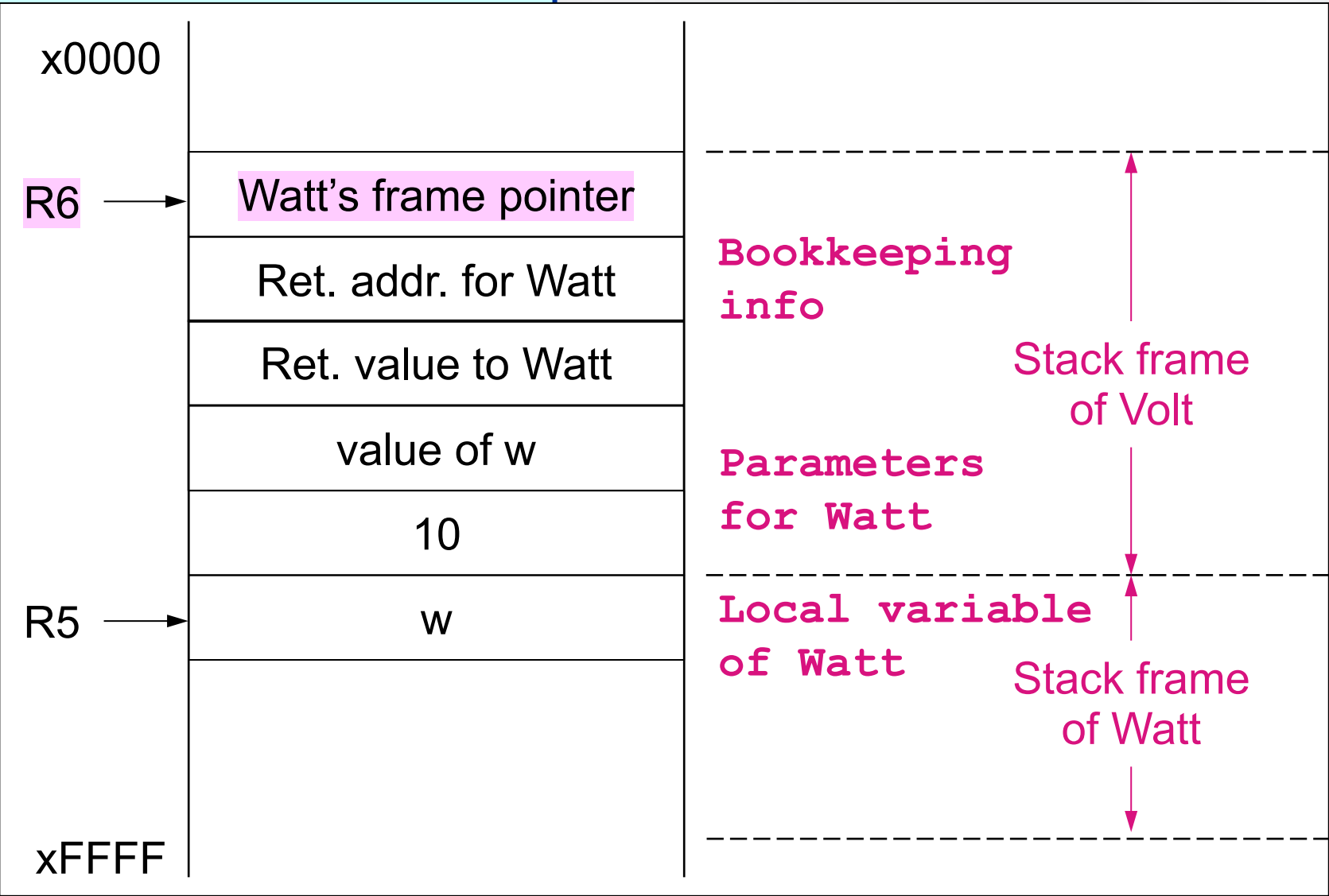
■ Four steps of the C-runtime stack protocol

- 2. The **callee** function pushes space for **local variables** and other info. onto the run-time stack, creating its **stack frame** on top of the stack

```
Volt:
  ADD R6,R6,#-1 ; alloc. spot for the
                  ; return value
  ADD R6,R6,#-1 ;
  STR R7,R6,#0  ; push R7 (

  ADD R6,R6,#-1
  STR R5,R6,#0  ; push R5
                  ; (caller's
  ADD R5,R6,#-1 ; set frame
  ADD R6,R6,#-2 ; alloc. me
                  ; Volt's lo
```

```
18
19 int Volt(int q, int r)
20 {
21     int k;
22     int m;
23
24     return k;
25 }
```



Function call in C

■ Four steps of the C-runtime stack protocol

2. The **callee** function pushes space for **local variables** and other info. onto the run-time stack, creating its **stack frame** on top of the stack

```

Volt:
    ADD R6,R6,#-1 ; alloc. sp
    ; return va

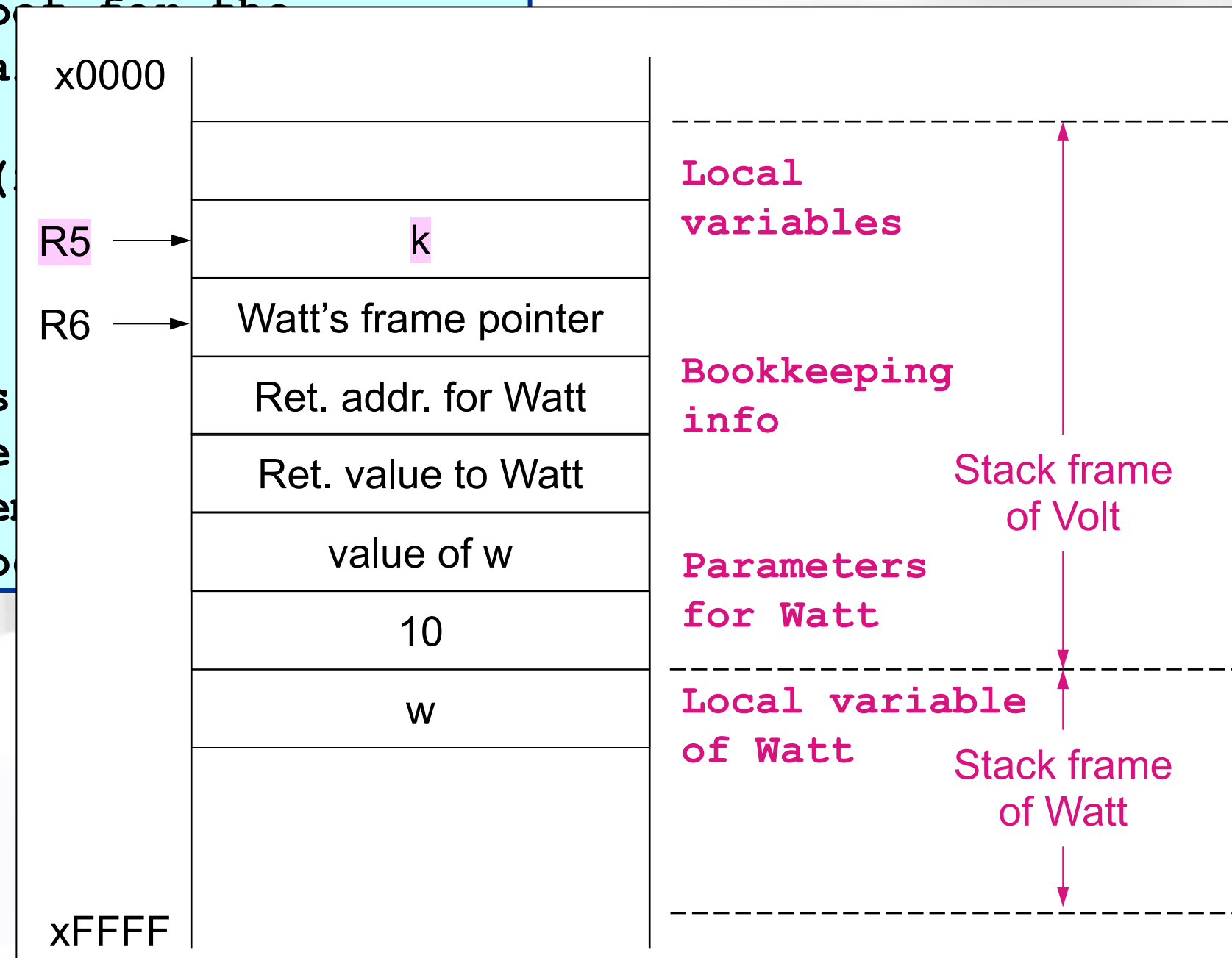
    ADD R6,R6,#-1 ;
    STR R7,R6,#0 ; push R7 (

    ADD R6,R6,#-1
    STR R5,R6,#0 ; push R5
    ; (caller's

    ADD R5,R6,#-1 ; set frame
    ADD R6,R6,#-2 ; alloc. me
    ; Volt's lo

```

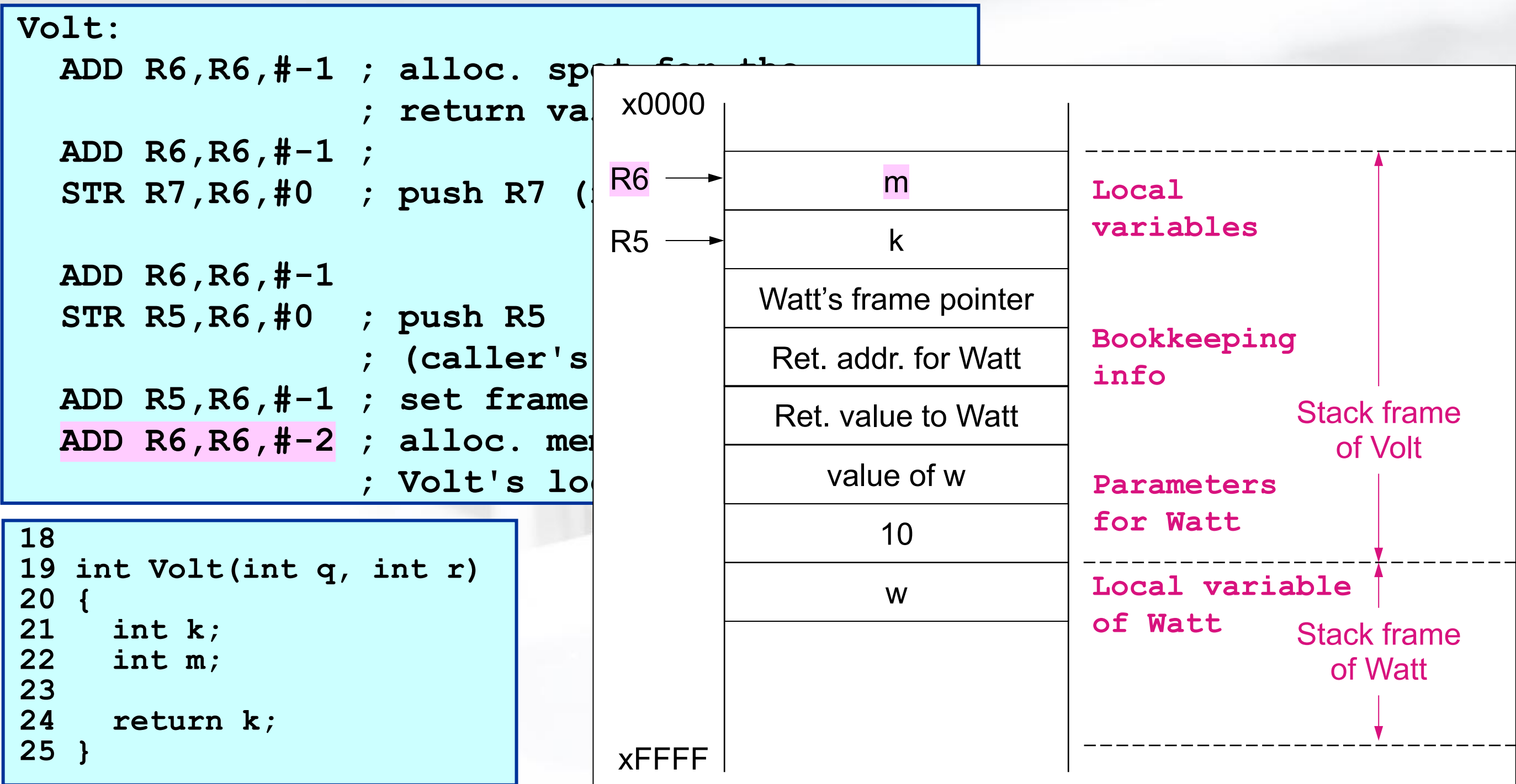
```
18
19 int Volt(int q, int r)
20 {
21     int k;
22     int m;
23
24     return k;
25 }
```





■ Four steps of the C-runtime stack protocol

- 2. The **callee** function pushes space for **local variables** and other info. onto the run-time stack, creating its **stack frame** on top of the stack



Function call in C

■ Four steps of the C-runtime stack protocol

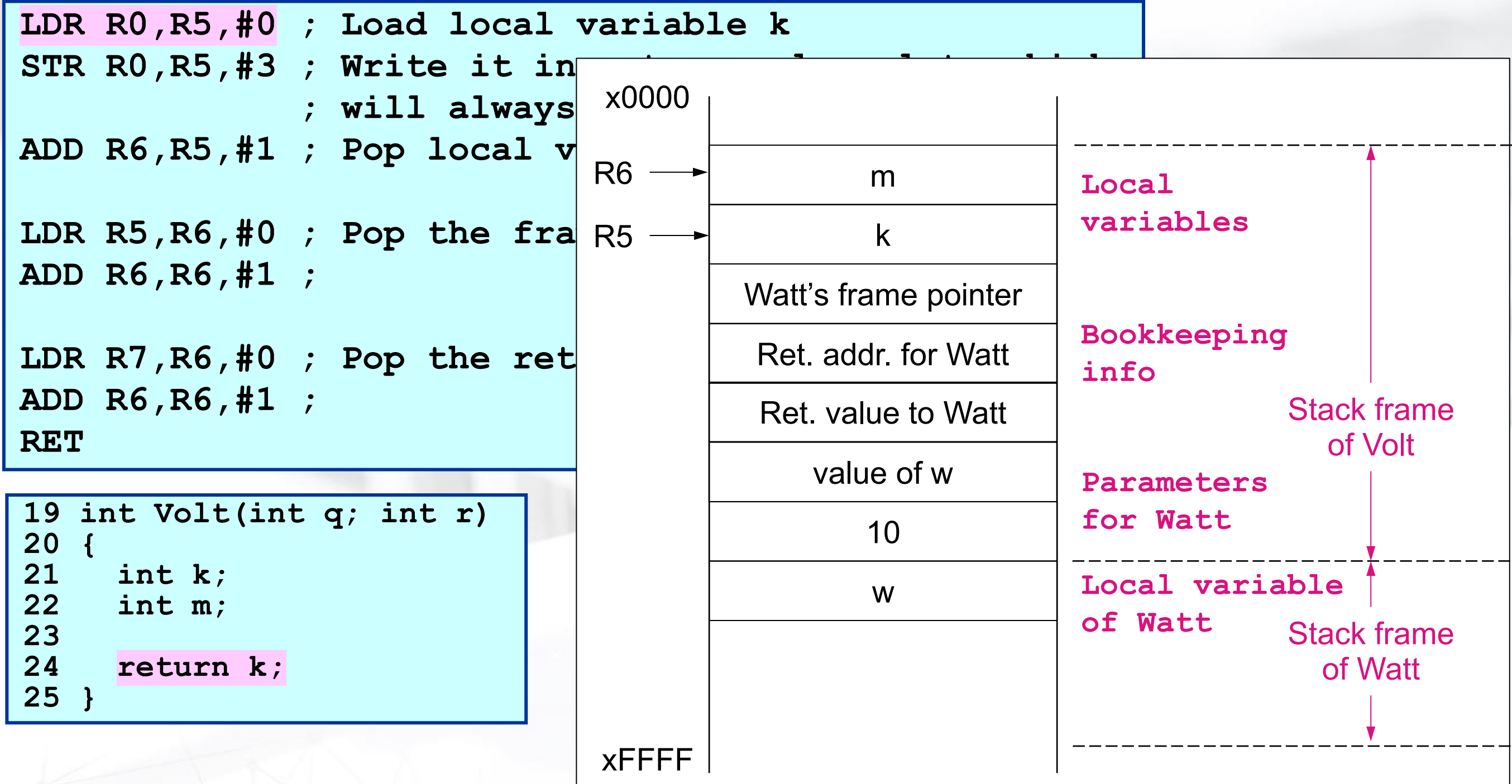
3. The **callee** executes the task

```
10 int Watt(int a)
11 {
12     int w;
13
14     w = Volt(w, 10);
15
16     return w;
17 }
18
19 int Volt(int q; int r)
20 {
21     int k;
22     int m;
23
24     return k;
25 }
```



■ Four steps of the C-runtime stack protocol

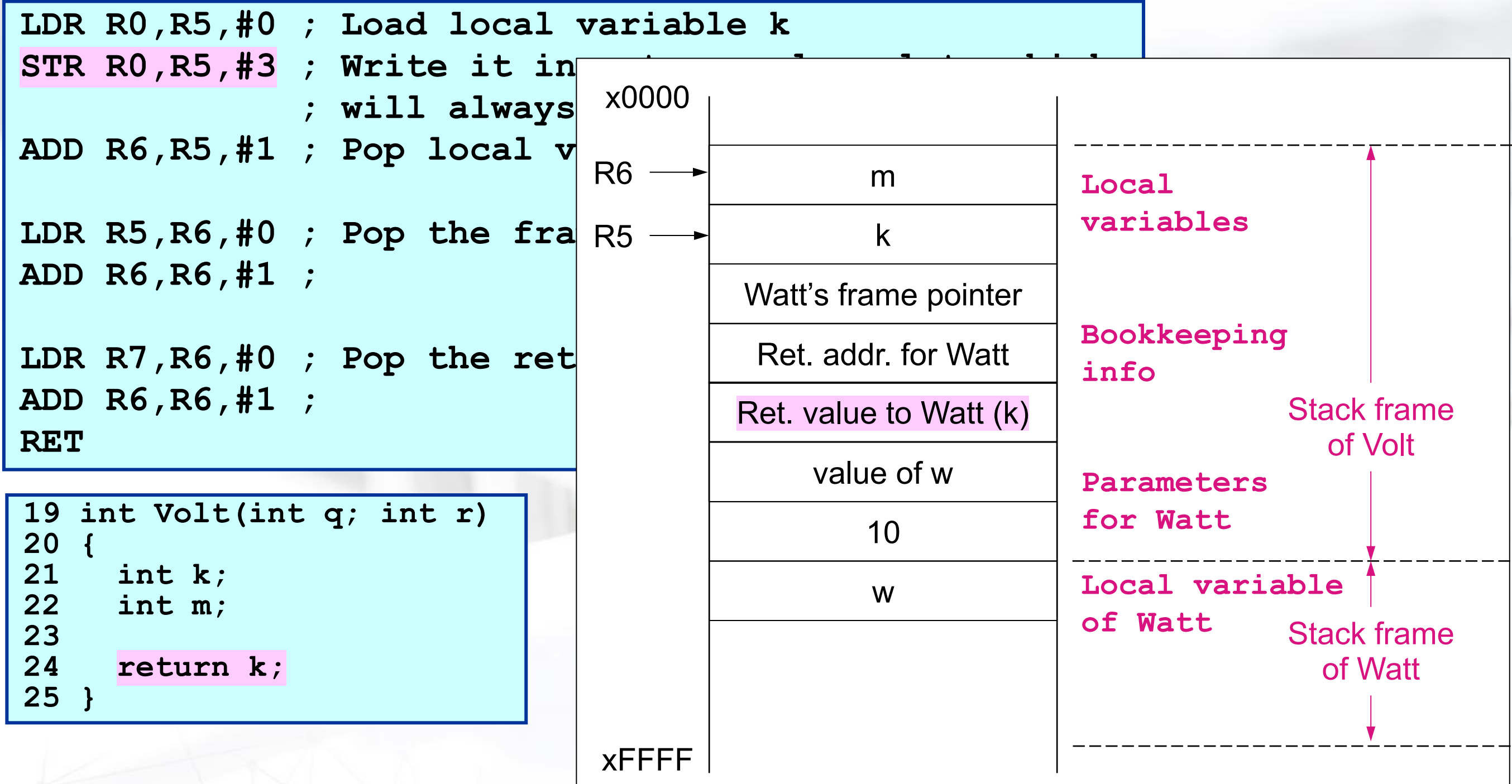
- 4. Once it is ready to return, the **callee** removes, or pops, its stack frame off the run-time stack and returns the **return value** and control to the caller





■ Four steps of the C-runtime stack protocol

- 4. Once it is ready to return, the **callee** removes, or pops, its stack frame off the run-time stack and returns the **return value** and control to the caller



Function call in C

■ Four steps of the C-runtime stack protocol

4. Once it is ready to return, the **callee** removes, or pops, its stack frame off the run-time stack and returns the **return value** and control to the caller

```
LDR R0,R5,#0 ; Load local variable k
```

```
STR R0,R5,#3 ; Write it in
```

```

; will always

```

```
ADD R6,R5,#1 ; Pop local v
```

```
LDR R5,R6,#0 ; Pop the fra
```

```
ADD R6,R6,#1 ;
```

```
LDR R7,R6,#0 ; Pop the ret
```

```
ADD R6,R6,#1 ;
```

RET

```
19 int Volt(int q; int r)
```

20 {

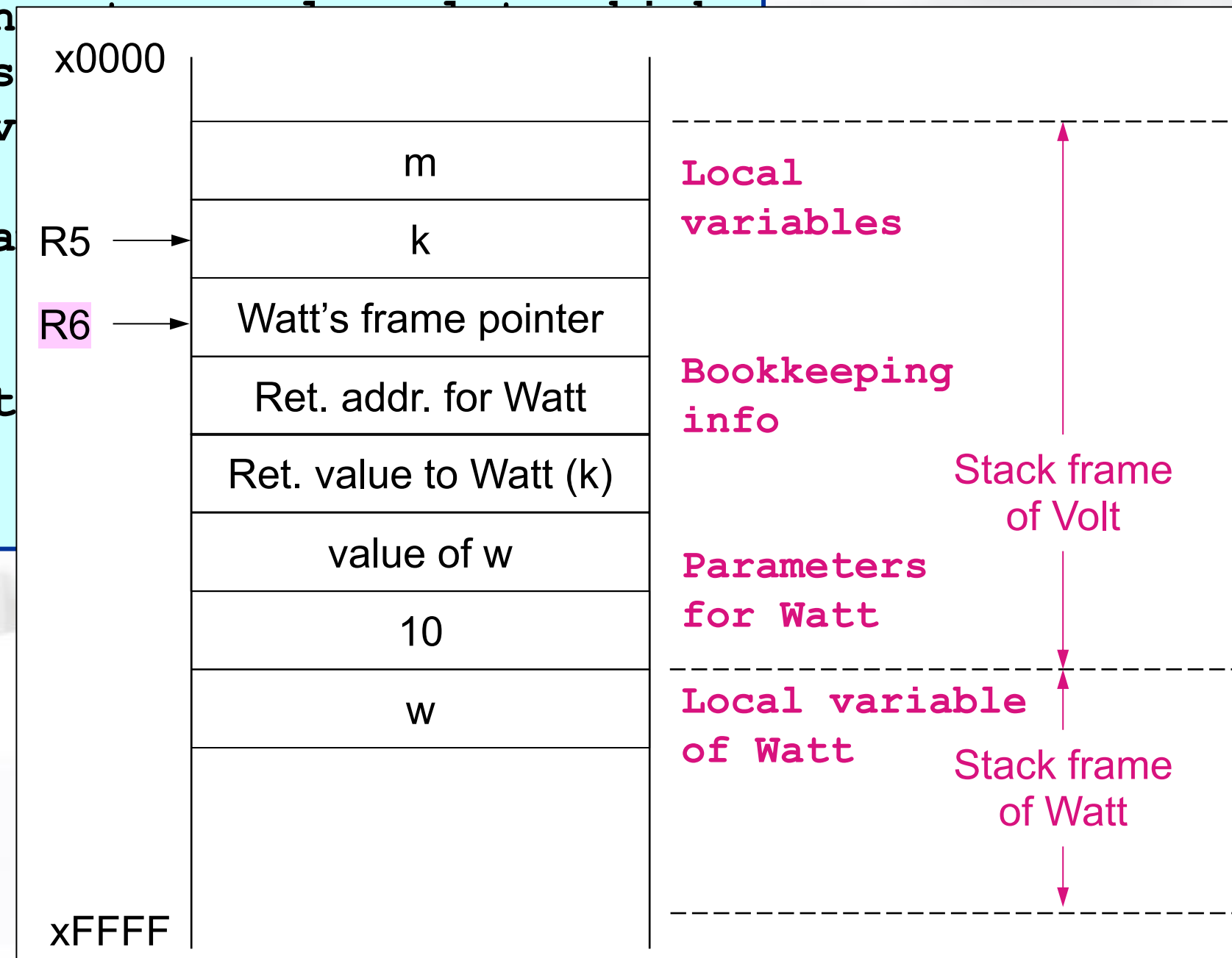
```
21     int k;
```

```
22     int m;
```

23

```
24 return k;
```

25 }



Function call in C

■ Four steps of the C-runtime stack protocol

4. Once it is ready to return, the **callee** removes, or pops, its stack frame off the run-time stack and returns the **return value** and control to the caller

```
LDR R0,R5,#0 ; Load local variable k
```

```
STR R0,R5,#3 ; Write it in
```

```
; will always
```

```
ADD R6,R5,#1 ; Pop local v
```

```
LDR R5,R6,#0 ; Pop the fra
```

```
ADD R6,R6,#1 ;
```

```
LDR R7,R6,#0 ; Pop the ret
```

```
ADD R6,R6,#1 ;
```

RET

```
19 int Volt(int q; int r)
```

20 {

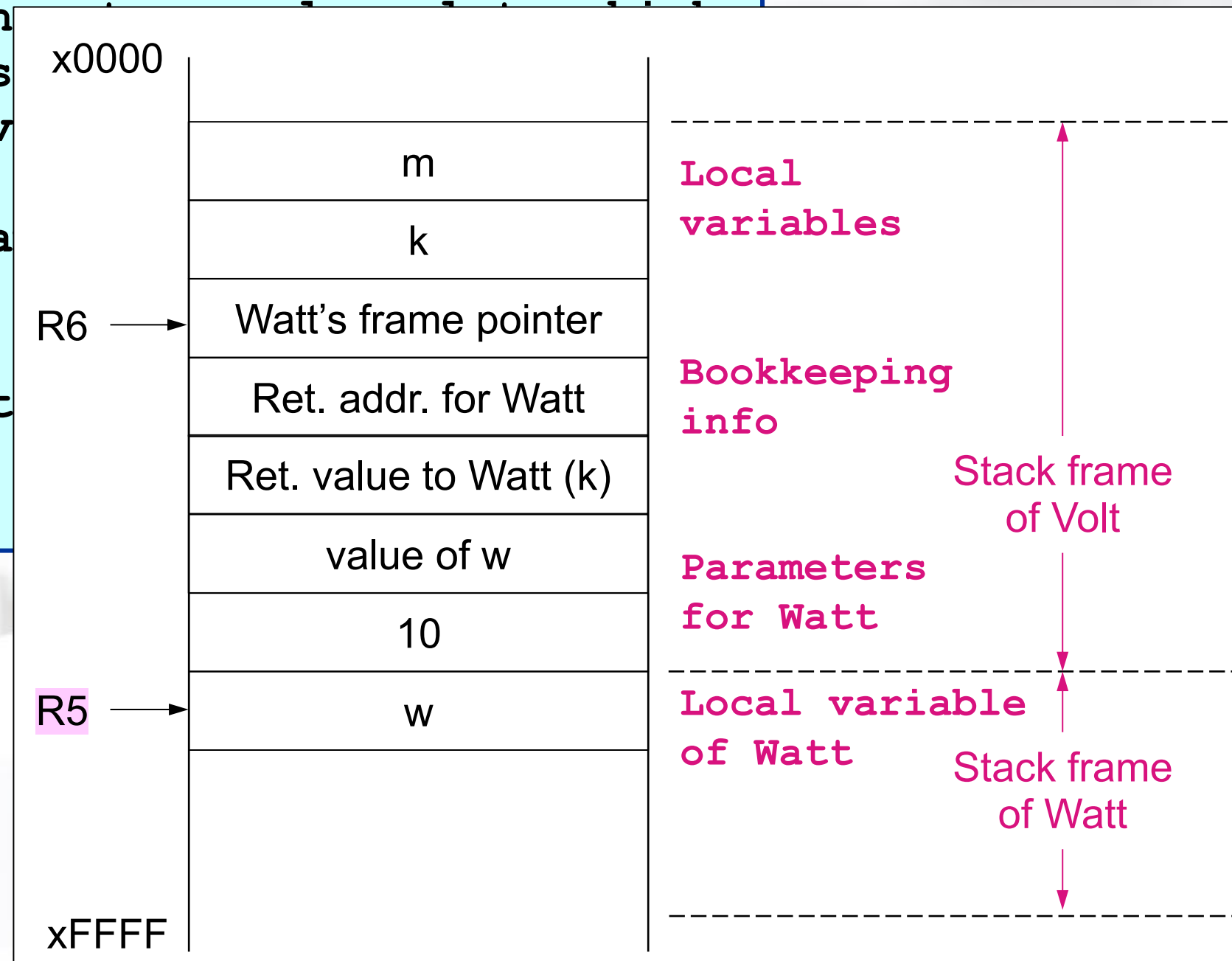
```
21    int k;
```

```
22     int m;
```

23

```
24 return k;
```

25 }



Function call in C

■ Four steps of the C-runtime stack protocol

4. Once it is ready to return, the **callee** removes, or pops, its stack frame off the run-time stack and returns the **return value** and control to the caller

```
LDR R0,R5,#0 ; Load local variable k
```

```
STR R0,R5,#3 ; Write it in
```

```
; will always
```

```
ADD R6,R5,#1 ; Pop local v
```

```
LDR R5,R6,#0 ; Pop the frame
```

```
ADD R6,R6,#1 ;
```

```
LDR R7,R6,#0 ; Pop the ret
```

```
ADD R6,R6,#1 ;
```

RET

```
19 int Volt(int q; int r)
```

20 {

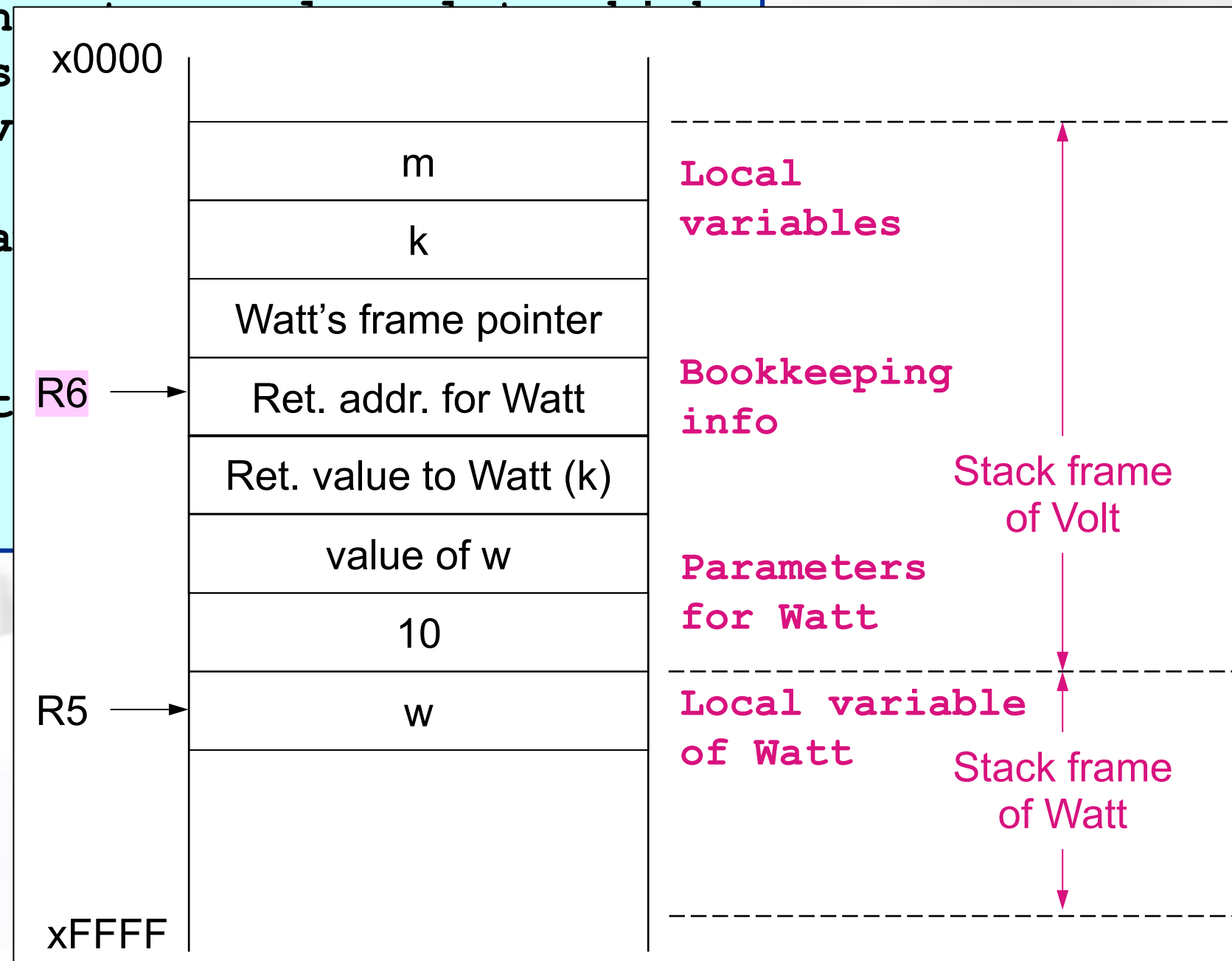
```
21  int k;
```

```
22     int m;
```

23

```
24 return k;
```

25 }



■ Four steps of the C-runtime stack protocol

4. Once it is ready to return, the **callee** removes, or pops, its stack frame off the run-time stack and returns the **return value** and control to the caller

```
LDR R0,R5,#0 ; Load local variable k
```

```
STR R0,R5,#3 ; Write it in
```

```
; will always
```

```
ADD R6,R5,#1 ; Pop local v
```

```
LDR R5,R6,#0 ; Pop the frame
```

```
ADD R6,R6,#1 ;
```

```
LDR R7,R6,#0 ; Pop the ret
```

```
ADD R6,R6,#1 ;
```

RET

```
19 int Volt(int q; int r)
```

20 {

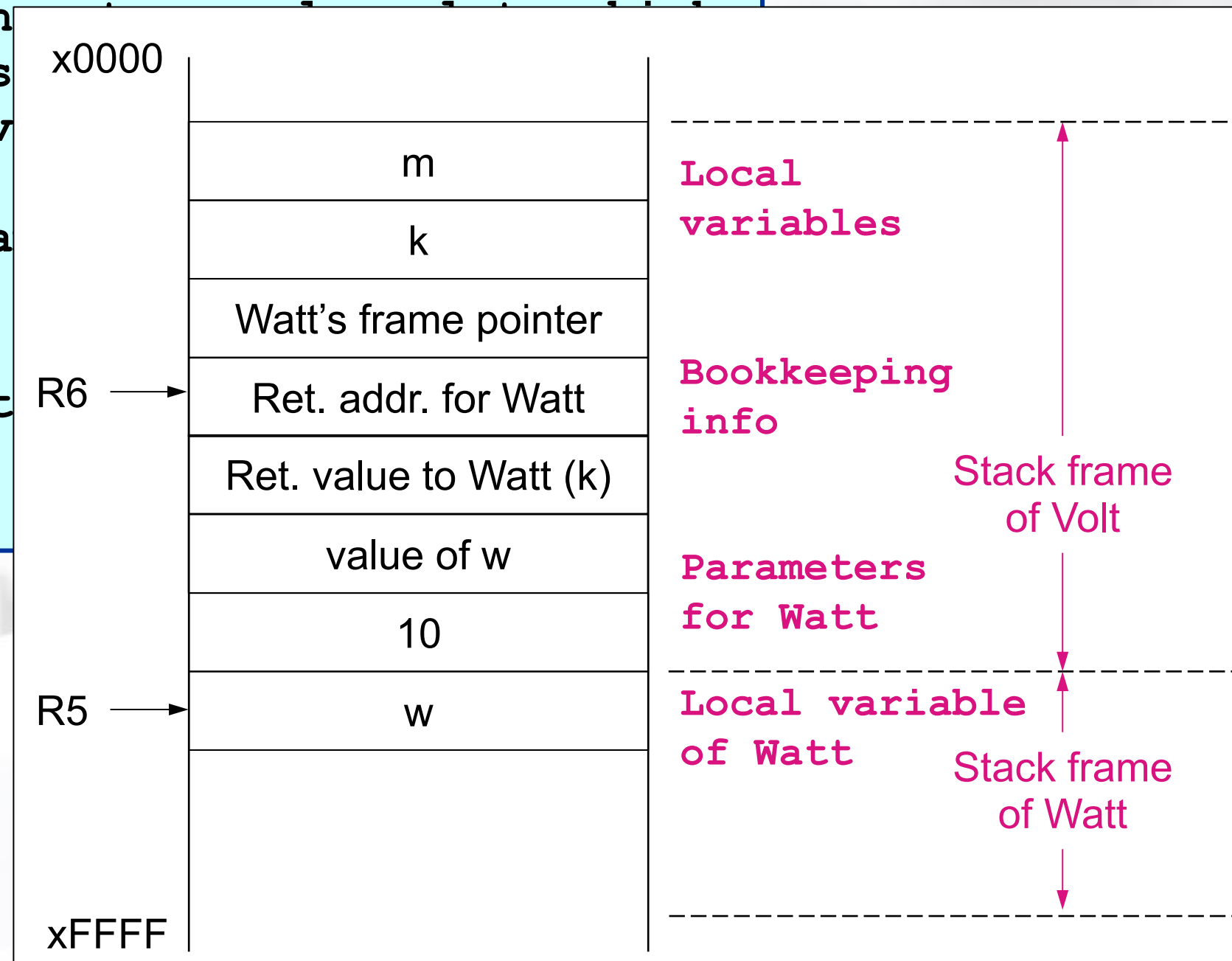
```
21  int k;
```

```
22     int m;
```

23

```
24 return k;
```

25 }



Function call in C

■ Four steps of the C-runtime stack protocol

4. Once it is ready to return, the **callee** removes, or pops, its stack frame off the run-time stack and returns the **return value** and control to the caller

```
LDR R0,R5,#0 ; Load local variable k
```

```
STR R0,R5,#3 ; Write it in
```

```
; will always
```

```
ADD R6,R5,#1 ; Pop local v
```

```
LDR R5,R6,#0 ; Pop the fra
```

```
ADD R6,R6,#1 ;
```

```
LDR R7,R6,#0 ; Pop the ret
```

```
ADD R6,R6,#1 ;
```

RET

```
19 int Volt(int q; int r)
```

20 {

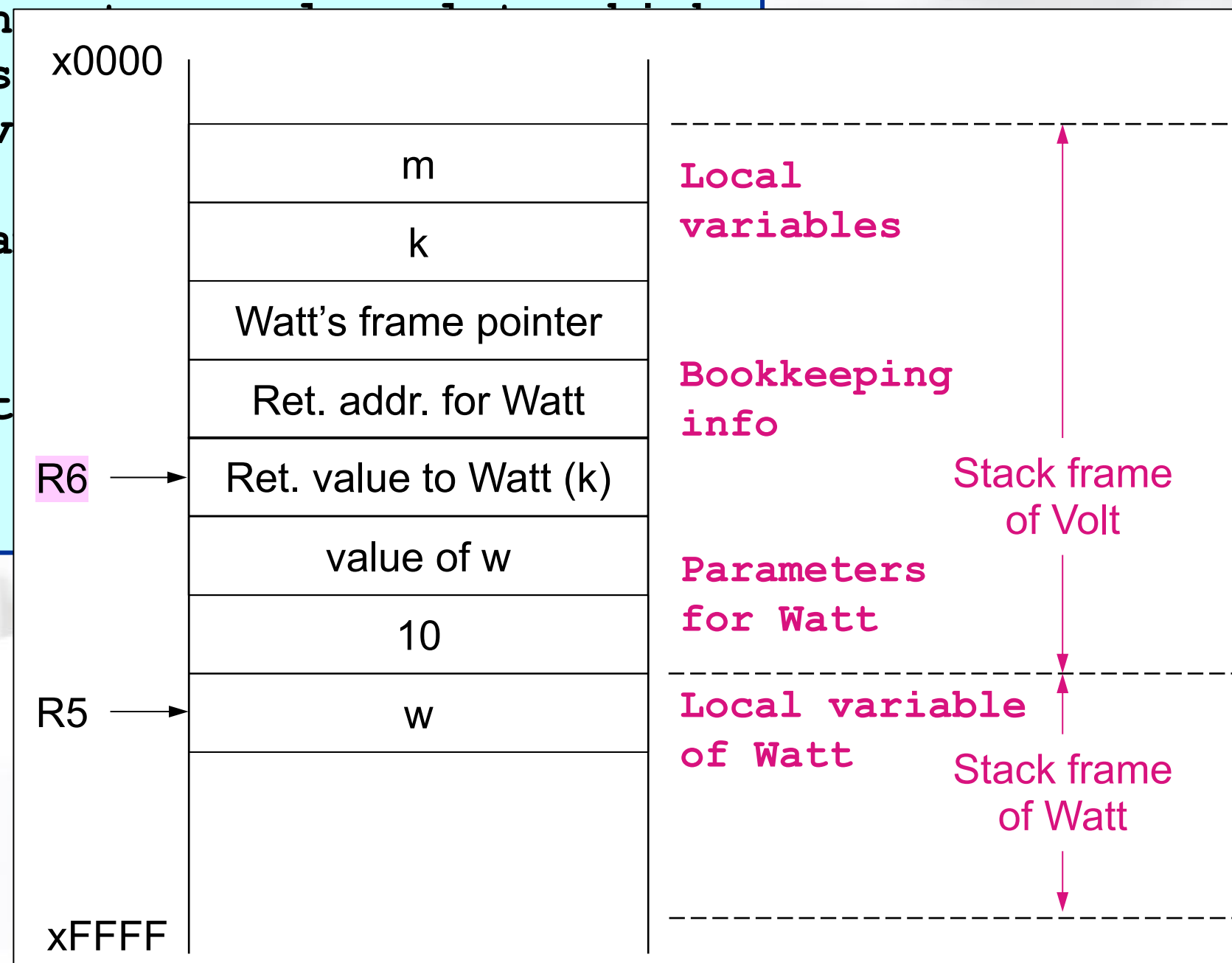
```
21     int k;
```

```
22     int m;
```

23

```
24 return k;
```

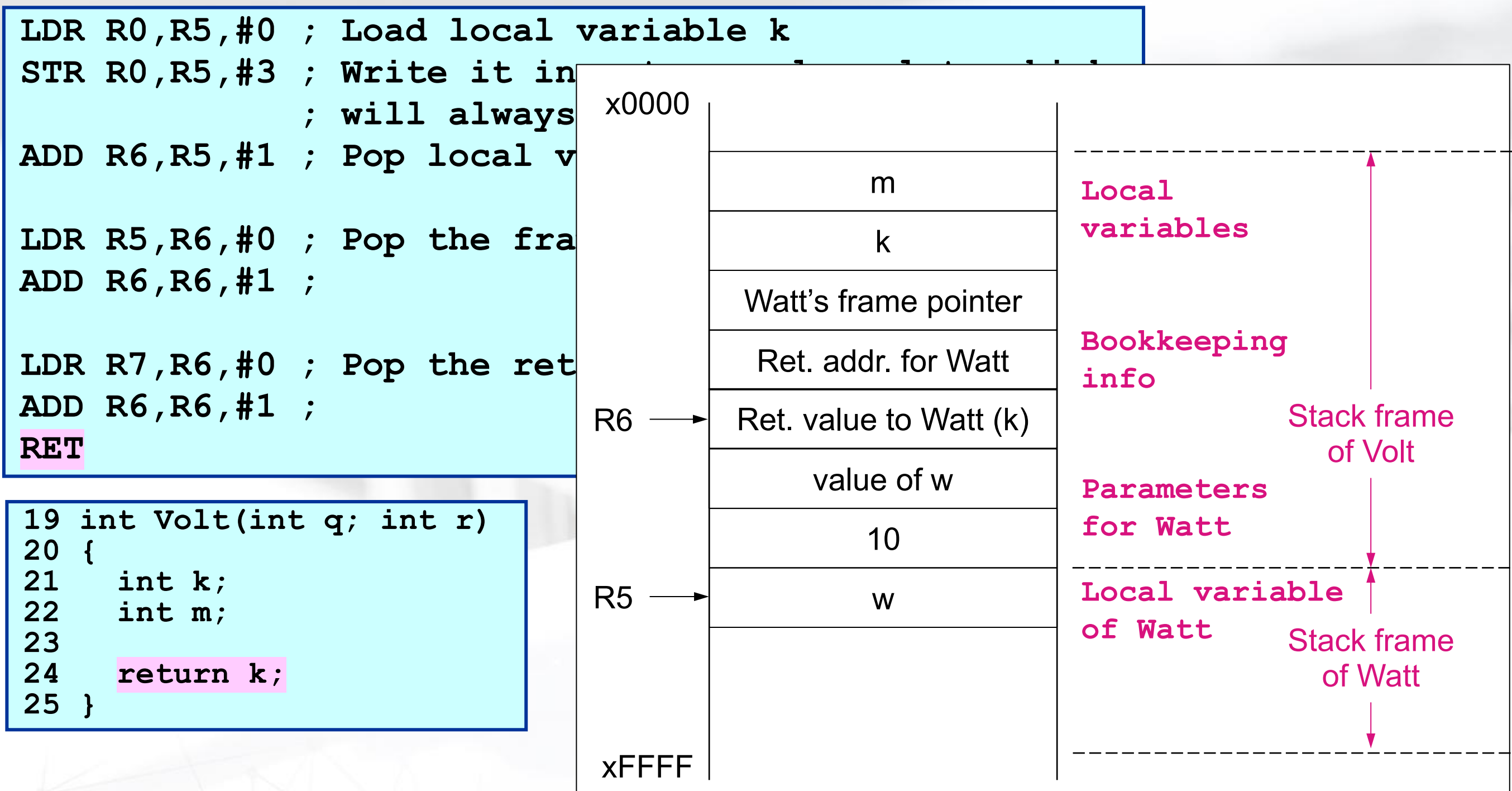
25 }





■ Four steps of the C-runtime stack protocol

- 4. Once it is ready to return, the **callee** removes, or pops, its stack frame off the run-time stack and returns the **return value** and control to the caller



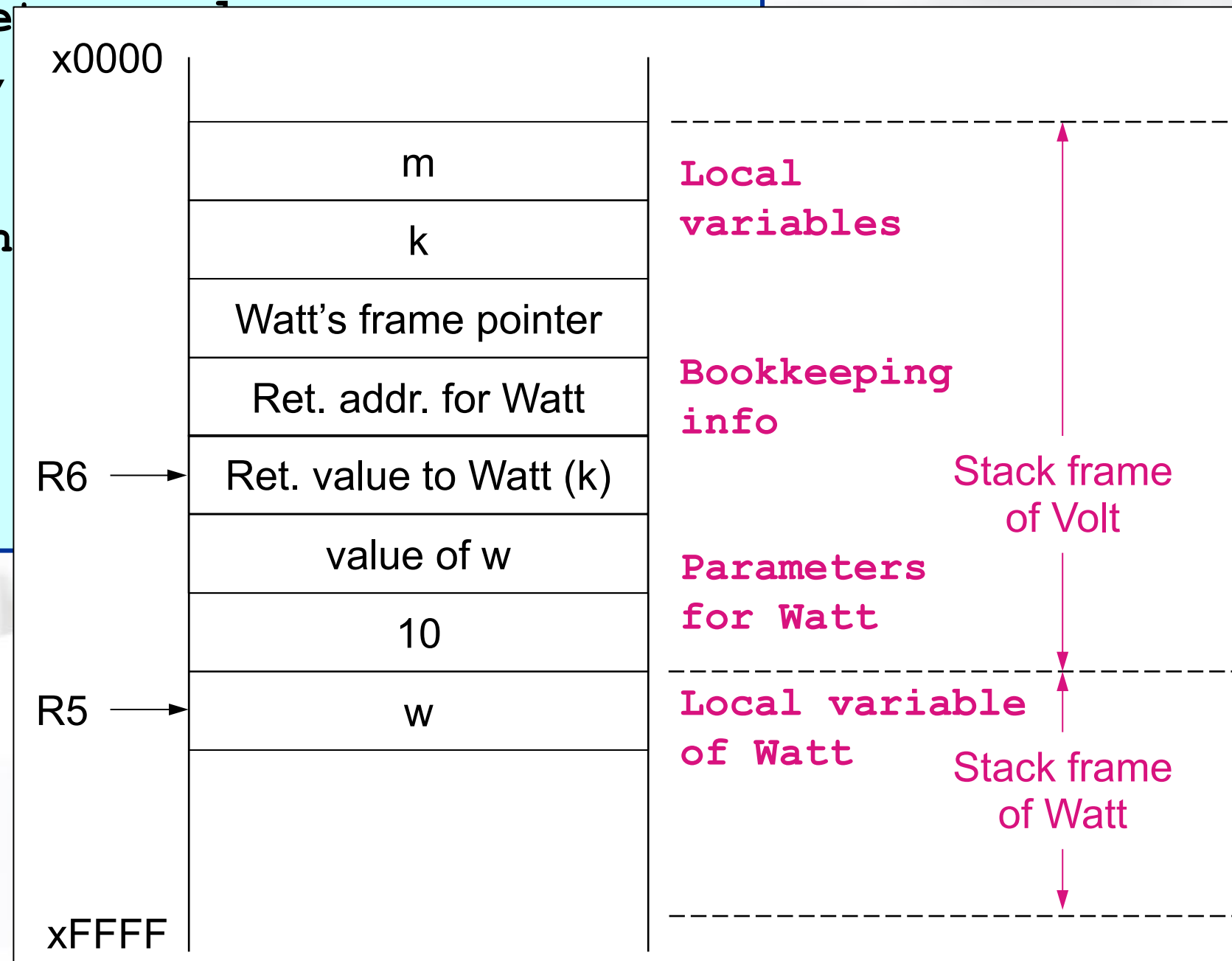
■ Four steps of the C-runtime stack protocol

4. Once it is ready to return, the **callee** removes, or pops, its stack frame off the run-time stack and returns the **return value** and control to the caller

```
JSR Volt
LDR R0,R6,#0 ; Load the re
STR R0,R5,#0 ; w = Volt(w,
ADD R6,R6,#1 ; Pop return

ADD R6,R6,#2 ; Pop argumen
```

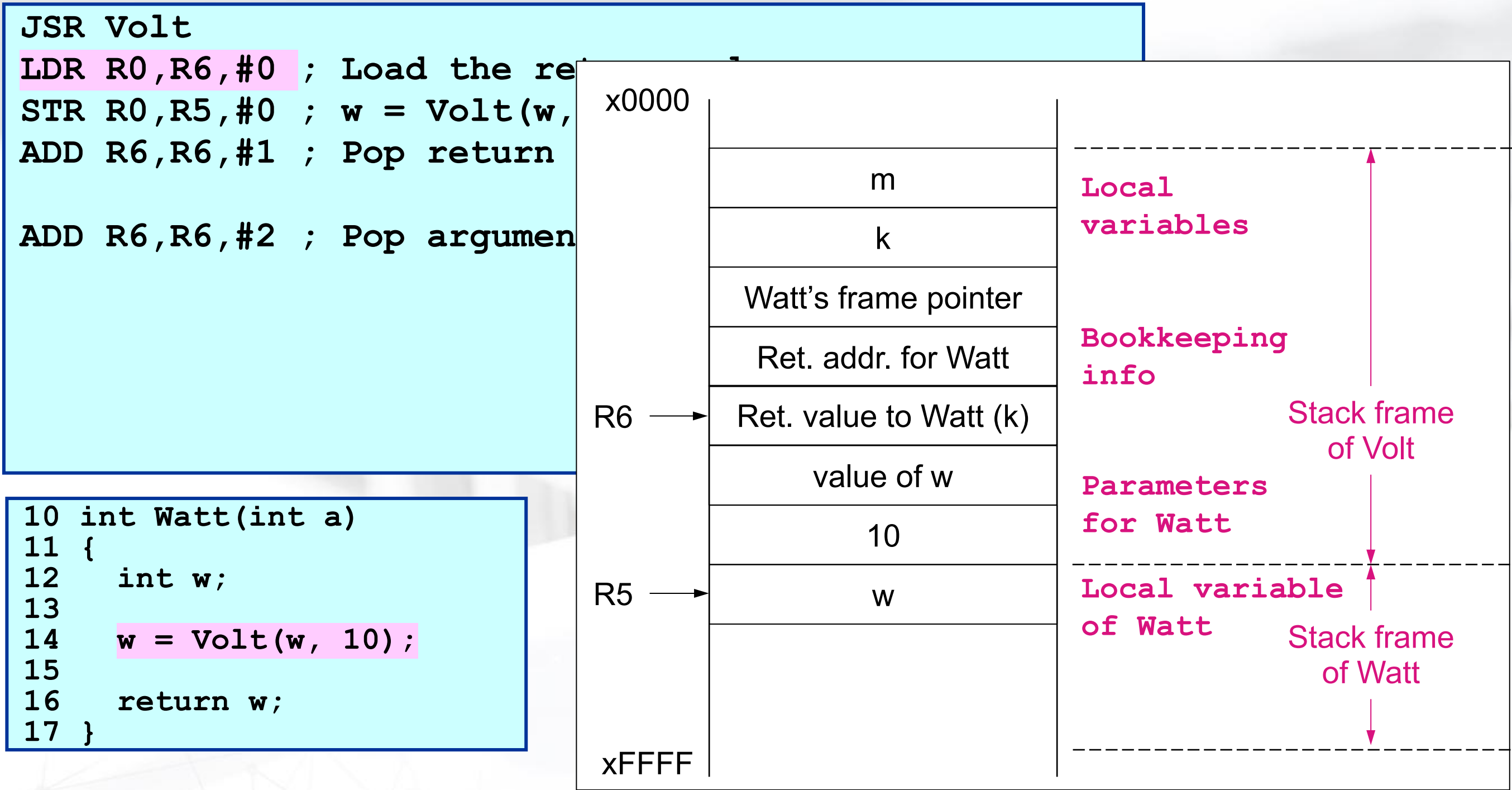
```
10 int Watt(int a)
11 {
12     int w;
13
14     w = Volt(w, 10);
15
16     return w;
17 }
```





■ Four steps of the C-runtime stack protocol

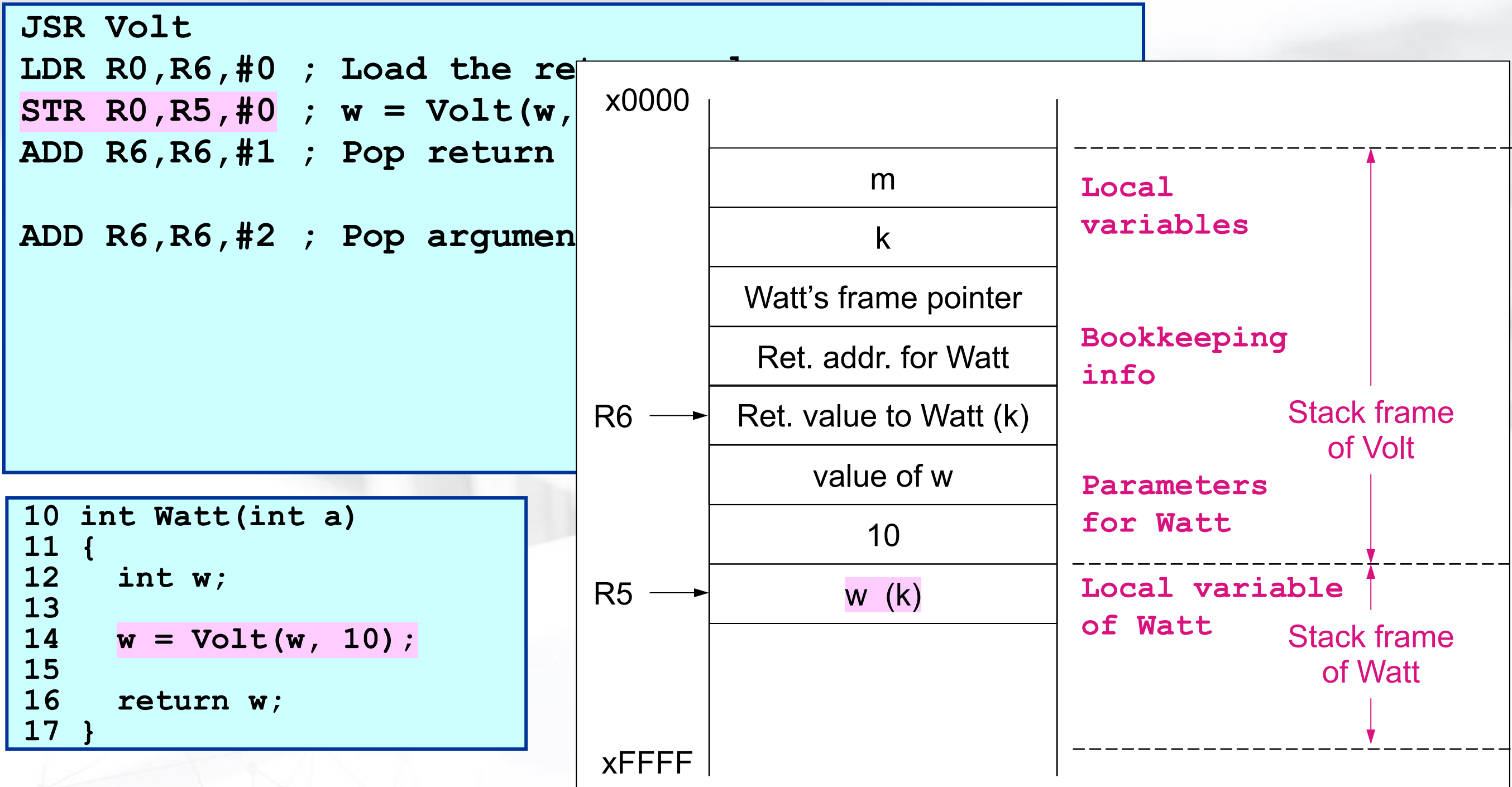
- 4. Once it is ready to return, the **callee** removes, or pops, its stack frame off the run-time stack and returns the **return value** and control to the caller





■ Four steps of the C-runtime stack protocol

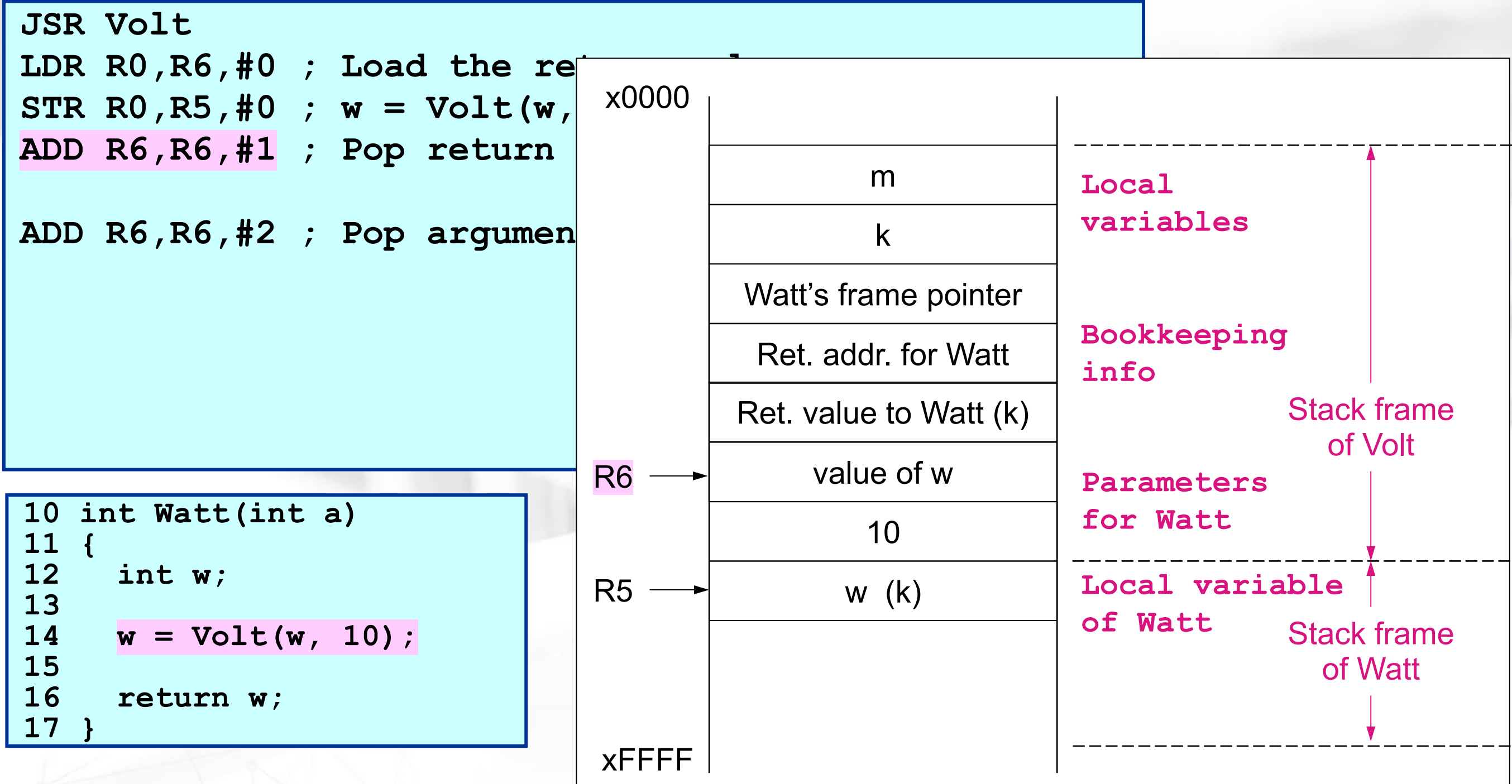
- 4. Once it is ready to return, the **callee** removes, or pops, its stack frame off the run-time stack and returns the **return value** and control to the caller





■ Four steps of the C-runtime stack protocol

- 4. Once it is ready to return, the **callee** removes, or pops, its stack frame off the run-time stack and returns the **return value** and control to the caller



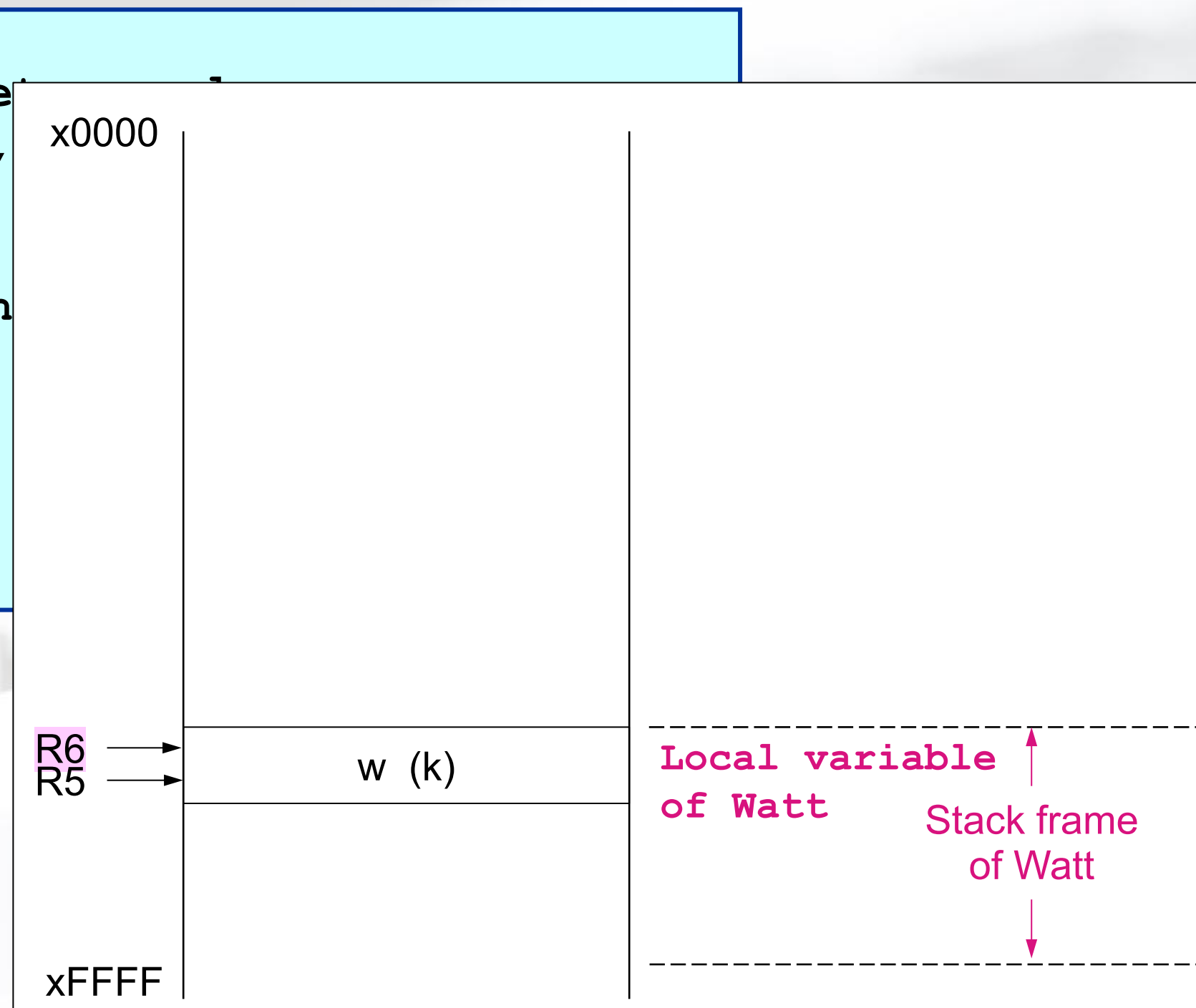
Function call in C

■ Four steps of the C-runtime stack protocol

4. Once it is ready to return, the **callee** removes, or pops, its stack frame off the run-time stack and returns the **return value** and control to the caller

```
JSR Volt
LDR R0,R6,#0 ; Load the re
STR R0,R5,#0 ; w = Volt(w,
ADD R6,R6,#1 ; Pop return
ADD R6,R6,#2 ; Pop argumen
```

```
10 int Watt(int a)
11 {
12     int w;
13
14     w = Volt(w, 10);
15
16     return w;
17 }
```



14.7 Following is the code for a C function named Bump.

```
int Bump(int x)
{
    int a;
    a = x + 1;
    return a;
}
```

- a. Draw the stack frame for Bump.
- b. Write one of the following in each entry of the stack frame to indicate what is stored there.
 - (1) Local variable
 - (2) Argument
 - (3) Address of an instruction
 - (4) Address of data
 - (5) Other
- c. Some of the entries in the stack frame for Bump are written by the function that calls Bump; some are written by Bump itself. Identify the entries written by Bump.

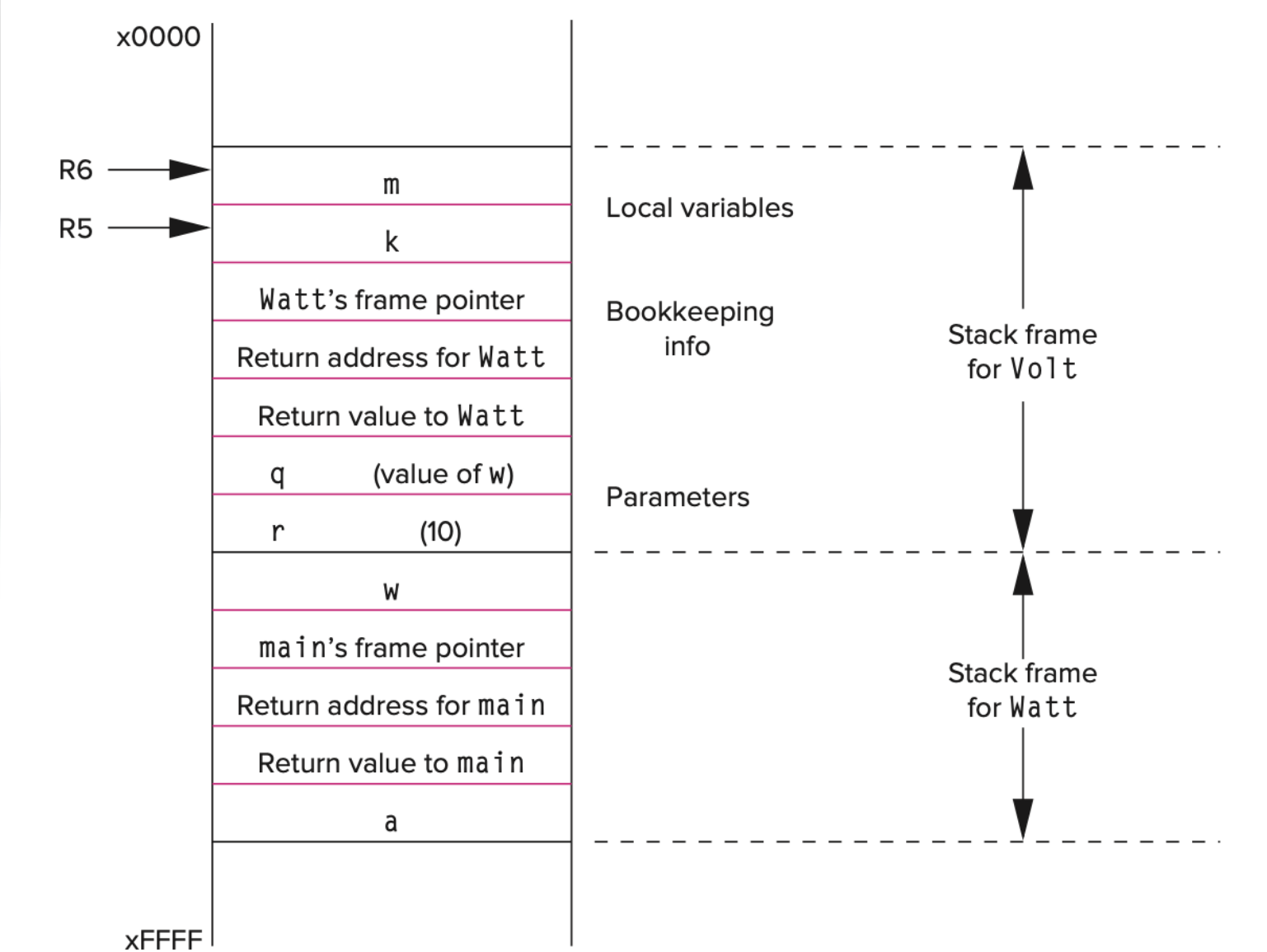


Figure 14.7 The run-time stack after the stack frame for Volt is pushed onto the stack.

14.15 The following C program is compiled into LC-3 machine language and loaded into address x3000 before execution. Not counting the JSRs to library routines for I/O, the object code contains three JSRs (one to function f, one to g, and one to h). Suppose the addresses of the three JSR instructions are x3102, x3301, and x3304. Also suppose the user provides 4 5 6 as input values. Draw a picture of the run-time stack, providing the contents of locations, if possible, when the program is about to return from function f. Assume the base of the run-time stack is location xEFFF.

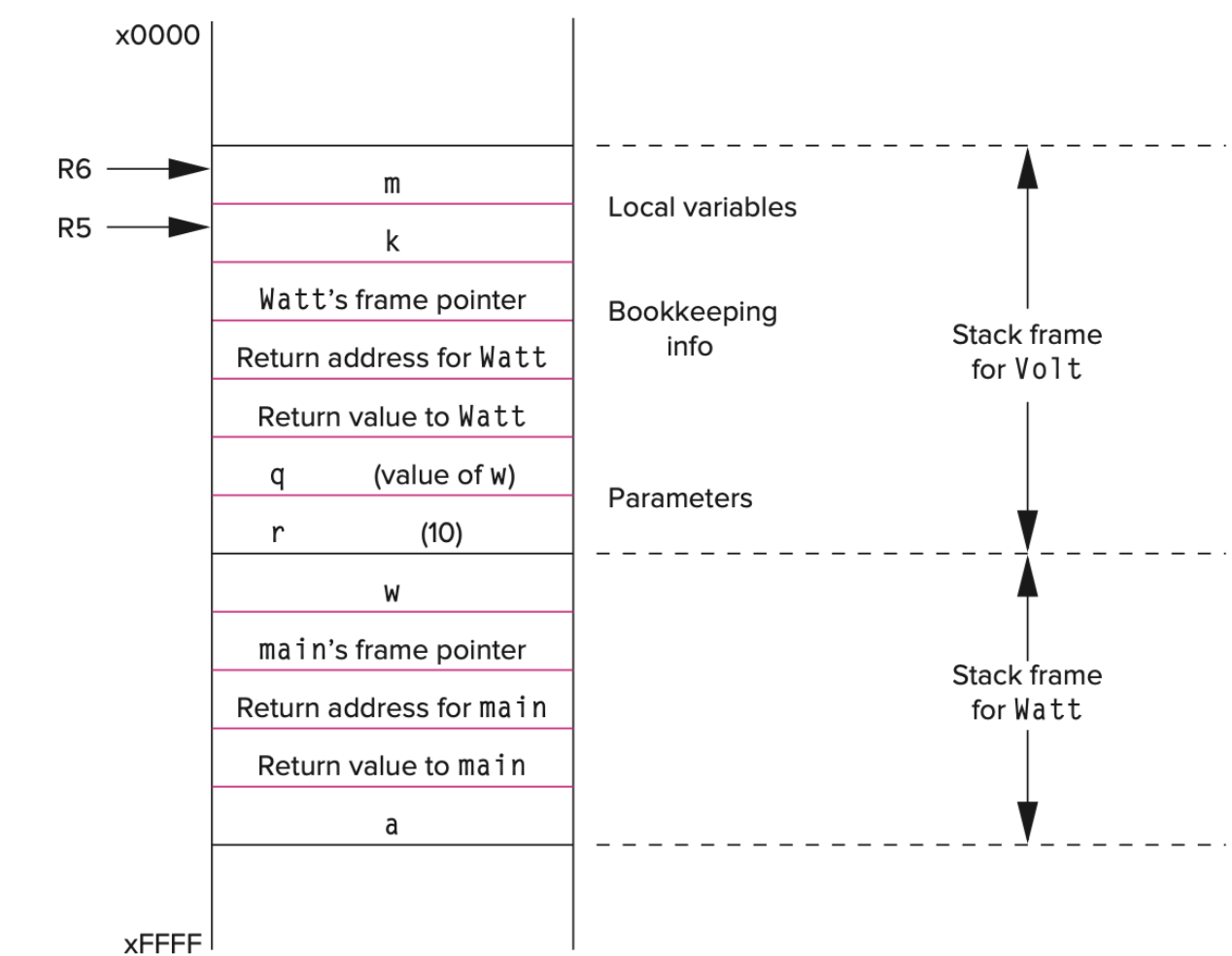


Figure 14.7 The run-time stack after the stack frame for Volt is pushed onto the stack.

```
#include <stdio.h>
int f(int x, int y, int z);
int g(int arg);
int h(int arg1, int arg2);

int main(void)
{
    int a, b, c;

    printf("Type three numbers: ");
    scanf("%d %d %d", &a, &b, &c);
    printf("%d", f(a, b, c));
}

int f(int x, int y, int z)
{
    int x1;

    x1 = g(x);
    return h(y, z) * x1;
}

int g(int arg)
{
    return arg * arg;
}

int h(int arg1, int arg2)
{
    return arg1 / arg2;
}
```